SR-404

C/C++

KLUDGE WAGON HAYWIRE · SERVICE MANUAL SUPPLEMENT

C/C++ Mistakes & References

Seven print-ready service cards: 81 bugs, each with a buggy snippet, what goes wrong, the fix, and the habits and tools that stop it from shipping. The referenced run records compiler diagnostics or program output. Checks cover the supplied examples and inputs; they do not prove every explanatory statement or all-input correctness.

Toolchain:
gcc (GCC) 16.2.1 20260810
clang version 23.1.3 (libFuzzer)

	SHEET	PAGE
01	**C Memory & Pointers** Buffer overflow, Off-by-one (fencepost), Use-after-free, Double free, Memory leak on an early return, Returning a pointer to a local, Uninitialized variable, NULL pointer dereference, realloc() result overwrites the only pointer, sizeof on a pointer, not the array, Stale pointer after realloc moved the block	2
02	**C Strings & I/O** strcpy into a fixed buffer, strncpy leaves no terminator, Missing null terminator, Format string from data, Mismatched printf specifiers, gets(), scanf without width or result check, Unchecked fopen / fprintf / fclose, Ignoring short writes, atoi() with no error checking, getchar() stored in a char, Comparing strings with ==	5
03	**C Integers, Floats & Undefined Behaviour** Signed integer overflow, Integer promotion: narrow unsigned may become signed, Unsigned wraparound in a countdown loop, Shift too far, or into the sign bit, Division by zero (and INT_MIN / -1), Integer division truncates, Strict aliasing (type punning via pointer cast), Unsequenced modifications (i++ + i++), Signed/unsigned comparison and silent narrowing, Comparing floats with ==, Out-of-range float → int conversion, Degrees passed to sin/cos, % keeps the sign of the dividend	8
04	**C Preprocessor & Everyday Slips** Macro without parentheses, Macro evaluates an argument twice, Multi-statement macro without do/while(0), Header without include guard, = instead of ==, Switch fall-through, Signal flag without volatile sig_atomic_t, Missing #include (implicit declaration), Missing semicolon, #if on an undefined macro, Misleading indentation / missing braces, Inverted or mis-negated condition, Global variable defined in a header	11
05	**C++ Objects & Ownership** Rule of 3/5 violation (shallow copy), Object slicing, Raw new/delete leaks on exceptions, Iterator invalidation, Dangling std::string_view, Missing virtual destructor, Throwing from a destructor, Assuming moved-from contents are unchanged, new[] paired with delete, Reference to a temporary through a function, shared_ptr reference cycle	14
06	**C++ Language Traps** Most vexing parse, Static initialisation order fiasco, ODR violation: same inline name, different body, Implicit conversion via a non-explicit constructor, std::vector<bool> is not a container of bool, auto deduction surprises, Lambda captures a local by reference, then outlives it, Data race on a shared counter, Virtual call from a constructor, map::operator[] inserts on lookup, std::move on a const object copies	17
07	**Concurrency & Embedded** Race condition: check-then-act, Deadlock from inconsistent lock order, volatile used as if it were atomic, Unsafe calls inside a POSIX signal handler, Publishing data with relaxed atomics, Unbounded wait: the watchdog bites, No input validation or output clamping, Trusting a length field (fuzz your parsers), Condition variable waited with if, not while, Tick-counter wraparound in timeouts	20

Example host debug build

```
gcc -std=c17 -Wall -Wextra -g -O0 \
    -fsanitize=address,undefined app.c
g++ -std=c++17 -Wall -Wextra -g \
    -fsanitize=address,undefined app.cpp
# threads: -fsanitize=thread (not with ASan)
# release: -O2 -D_FORTIFY_SOURCE=3 -Werror
```

Reading the cards

Entries have stable IDs, without unsupported frequency rankings. The ▸ line summarizes the referenced compiler/run capture; raw commands and diagnostics remain in verification_report.json. Source references name their document version and section where supplied. "SR-404" names the Kludge Wagon system where you'll meet the bug in the game. The IDs (C-MEM-03, CPP-TRAP-07, ...) match the game's mistakes catalog.

SHEET 01

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Memory & Pointers

Array bounds, owned allocations and object lifetimes, with bounded compiler and runtime captures.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 11 entries

C-MEM-01 Buffer overflow

SR-404 · Dashboard: RPM history

× BUGGY
```c
int rpm_log[8];
void log_rpm(int i, int v) {
    rpm_log[i] = v; // index8 is outside the permitted elements
}
```

▶ ASan: global-buffer-overflow

What goes wrong: This assignment requests element 8 of an eight-element array, violating its bounds. ISO C does not require a particular overwrite, crash or gauge response. Implementations and instrumentation can add checks; the language contract still requires a valid index.

> FIX
```c
#define RPM_LOG_LEN 8
int rpm_log[RPM_LOG_LEN];
bool log_rpm(size_t i, int v) {
    if (i >= RPM_LOG_LEN) return false;  // reject, don't corrupt
    rpm_log[i] = v;
    return true;
}
```

Sources: C11 draft 6.5.2.1p2;6.5.6p8;6.3.1.3; GCC14 docs Warray-bounds; GCC14 docs Wanalyzer-out-of-bounds; Clang docs Introduction;Usage;Limitations; Memory host example capture C-MEM-01 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-01 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Use an actual length and validate external signed values before converting to a size type. A size_t parameter alone is not validation. GCC14 documents selected bounds/analyzer warnings; ASan checks instrumented executed accesses. Neither a warning-free build nor a sample pass proves every index safe.

C-MEM-02 Off-by-one (fencepost)

SR-404 · Dashboard: gauge reset

× BUGGY
```c
void reset_gauges(int g[4]) {
    for (int i = 0; i <= 4; i++)   // 5 iterations
        g[i] = 0;
}
```

▶ ASan: stack-buffer-overflow

What goes wrong: With the shown zero start and unit increment, the inclusive condition requests index 4 from a four-element caller array. That access is invalid. Inclusive comparisons and length-plus-one storage are not intrinsically wrong; the actual element range and terminating-null capacity matter.

> FIX
```c
void reset_gauges(int *g, size_t n) {
    for (size_t i = 0; i < n; i++)  // half-open range [0, n)
        g[i] = 0;
}
/* call: reset_gauges(g, sizeof g / sizeof g[0]); */
```

Avoid it: A half-open loop is a useful policy when n is the actual element count. Ordinary int g[4] in this parameter adjusts to int*, carrying no array length. Caller capacity still matters. The supplied ASan capture has tool/path/symbolization scope; no exact-line diagnostic is guaranteed for every build.

Sources: C11 draft 6.7.6.3p7;6.8.5.3;6.5.6;6.7.9p14; Clang docs Usage;Limitations; Memory host example capture C-MEM-02 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-02 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-03 Use-after-free

SR-404 · Dashboard: warning text

× BUGGY
```c
char *msg = malloc(16);
if (!msg) return 1;
snprintf(msg, 16, "LOW FUEL");
free(msg);
puts(msg);            // invalid lifetime access
```

▶ gcc -Wuse-after-free=: pointer 'msg' used after 'free'

What goes wrong: free ends the allocated object lifetime and makes space available for later allocation. Old pointer values become indeterminate. Access does not have a portable stale-data or heap-corruption result, regardless of whether physical bytes appear unchanged.

> FIX
```c
char *msg = malloc(16);
if (msg == NULL) return 1;
int rc = 0;
if (snprintf(msg, 16, "LOW FUEL") < 0 || puts(msg) == EOF)
    rc = 1;
free(msg);
msg = NULL;  // clear this handle; other aliases need their own policy
return rc;
```

Sources: C11 draft 6.2.4p2;7.22.3.3;7.21.6.5;7.21.7.9; GCC14 docs Wuse-after-free;Wall; GCC14 docs Wanalyzer-use-after-free; Clang docs Introduction;Limitations; Memory host example capture C-MEM-03 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-03 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Release after intended last use and track ownership/borrowing explicitly. Clearing one handle does not repair other aliases. GCC14 -Wuse-after-free and analyzer checks request selected diagnostics; ASan does not catch every possible lifetime error.

C-MEM-04 Double free

SR-404 · ECU shutdown

× BUGGY
```c
void ecu_shutdown(char *buf) { free(buf); }
int main(void) {
    char *buf = malloc(32);
    if (buf == NULL) return 1;
    ecu_shutdown(buf);
    free(buf);  // old allocation is no longer live
}
```

▶ ASan: attempting double-free

What goes wrong: Releasing already-deallocated nonnull storage is undefined. free(NULL) is a separate, permitted no-op. A specific allocator corruption, abort message or later duplicate allocation is not an ISO C guarantee.

> FIX
```c
void ecu_shutdown(char **owner) {
    if (owner == NULL) return;
    free(*owner);
    *owner = NULL;  // clear the supplied owner handle
}
int main(void) {
    char *buf = malloc(32);
    if (buf == NULL) return 1;
    ecu_shutdown(&buf);
    ecu_shutdown(&buf);  // this call sees NULL
    free(buf);           // free(NULL) does nothing
    return 0;
}
```

Sources: C11 draft 7.22.3.3p2;6.2.4p2; Clang docs Double-free checks; GCC14 docs Wanalyzer-double-free; Memory host example capture C-MEM-04 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-04 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: A single release responsibility is an author ownership policy, not a C alias-safety type system. Document any shared/borrowed views separately. The named ASan double-free capture and GCC14 -Wanalyzer-double-free check are bounded diagnostic mechanisms.

C-MEM-05 Memory leak on an early return

SR-404 · Fuel-injection map (Ep 9)

✕ BUGGY

```
int load_fuel_map(int ok) {
    double *map = malloc(64 * sizeof *map);
    if (!map) return -1;
    if (!ok) return -1;      // early return leaks map
    map[0] = 14.7;           /* ... use the map ... */
    free(map);
    return 0;
}
```

▸ LeakSanitizer: detected memory leaks

What goes wrong: After a successful allocation, the shown !ok return loses its only program reference without releasing the map. An early return can instead follow allocation failure, cleanup or an ownership transfer. Exhaustion timing and a particular ECU stall require the actual allocation/model/runtime contract.

✓ FIX

```
int load_fuel_map(int ok) {
    int rc = -1;
    double *map = NULL;
    const size_t count = 64;
    if (count > SIZE_MAX / sizeof *map) goto out;
    map = malloc(count * sizeof *map);
    if (map == NULL || !ok) goto out;
    map[0] = 14.7;  /* exercise use */
    rc = 0;
out:
    free(map);  // includes the permitted NULL case
    return rc;
}
```

Sources: C11 draft 7.22.3;7.22.3.3;6.2.5p9;6.8.6.1; Clang docs Memory leak detection; Valgrind manual 4.2.10;4.3 leak-check; GCC14 docs Wanalyzer-malloc-leak; Memory host example capture C-MEM-05 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-05 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: A cleanup label is one way to release resources on each path; correct multiple-exit designs also exist. Clang ASan documents integrated leak checking on supported Linux configurations. Valgrind leak-check and GCC14 analyzer checks have their own options and limits; they are not all-input ownership proofs.

C-MEM-06 Returning a pointer to a local

SR-404 · Transmission: gear readout

✕ BUGGY

```
const char *gear_name(int g) {
    char buf[8];
    snprintf(buf, sizeof buf, "G%d", g);
    return buf;            // buf's lifetime ends here
}
```

▸ gcc -Wreturn-local-addr: function returns address of local variable

What goes wrong: buf has automatic storage duration. Its lifetime ends on leaving this function, so the returned old pointer value becomes indeterminate. C does not require a stack layout, physical byte erasure or overwrite by the next function call.

✓ FIX

```
int gear_name(int g, char *buf, size_t n) {
    if (buf == NULL || n == 0) return -1;
    int written = snprintf(buf, n, "G%d", g);
    return written < 0 || (size_t)written >= n ? -1 : 0;
}
/* char name[8]; if (!gear_name(3, name, sizeof name))
puts(name); */
```

Avoid it: Caller storage must remain live and have the stated capacity. This example reports formatting/truncation failure; the buffer may already be modified on failure. By-value results and static-duration constant tables are other design choices. Resolve GCC14 return-local-address warnings in their actual source context.

Sources: C11 draft 6.2.4p2-p6;7.21.6.5; GCC14 docs Wno-return-local-addr; Memory host example capture C-MEM-06 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-06 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-07 Uninitialized variable

SR-404 · Speed limiter

✕ BUGGY

```
int clamp_speed(int raw) {
    int out;
    if (raw > 0)
        out = raw;
    return out;          // indeterminate when raw <= 0
}
```

▸ gcc -Wanalyzer-use-of-uninitialized-value: use of uninitialized valu…

What goes wrong: Uninitialized automatic out has an indeterminate value. On raw<=0 this return uses the uninitialized register-eligible int and violates C11. The rule is not a description of leftover stack bytes or a promise of changing values between builds.

✓ FIX

```
int clamp_speed(int raw) {
    int out = 0;         // defined default
    if (raw > 0)
        out = raw;
    return out;
}
```

Avoid it: Choose a source-level default that matches the intended logic; zero is this exercise policy. GCC14, Clang diagnostics, MemorySanitizer and Valgrind provide selected checks with analysis/instrumentation requirements. GCC14 documents that auto-var-init hardening still permits uninitialized-source warnings and optimization; it does not replace the missing logical initialization.

Sources: C11 draft 6.7.9p10;6.3.2.1p2;6.2.6.1; GCC14 docs Wuninitialized;Wmaybe-uninitialized; GCC14 docs Wanalyzer-use-of-uninitialized-value; Clang docs Wsometimes-uninitialized; Clang docs Introduction;Instrumentation;Limitations; Valgrind manual 4.2.2; GCC14 docs ftrivial-auto-var-init; Clang argument docs ftrivial-auto-var-init; Memory host example capture C-MEM-07 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-07 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-08 NULL pointer dereference

SR-404 · 4WD transfer case (Ep 8)

✕ BUGGY

```
const struct mode *find_mode(const char *n);  // NULL if unknown
void engage(const char *n) {
    printf("ratio %d\n", find_mode(n)->ratio);  // "4LO": NULL!
}
```

▸ UBSan: member access within null pointer of type 'const struct mode'

What goes wrong: This fixture returns NULL for an unknown mode; other APIs have their own failure representations. Access through a null pointer is undefined, with no required physical fault. Compiler null-check optimization depends on the target and options.

✓ FIX

```
const struct mode *find_mode(const char *n); // fixture contract
int engage(const char *n) {
    if (n == NULL) return -1;
    const struct mode *m = find_mode(n);
    if (m == NULL) return -1;
    return printf("ratio %d\n", m->ratio) < 0 ? -1 : 0;
}
```

Sources: C11 draft 6.5.3.2p4;6.5.2.3;7.21.6.1p14; GCC14 docs fdelete-null-pointer-checks; GCC14 docs Wnull-dereference; GCC14 docs Wanalyzer-null-dereference; Clang docs null; Memory host example capture C-MEM-08 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-08 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Read each API contract, including borrowed-object lifetime and failure status. This fixture only checks a lookup and prints a ratio; it does not prove a vehicle-mode transition. GCC14 analyzer/null warnings and UBSan document selected checks, not complete validation of every pointer or path.

C-MEM-09 realloc() result overwrites the only pointer

SR-404 · Trip computer log

✕ BUGGY

```
static char *trip_log; // driver initially allocates64 bytes
int grow_log(size_t want) {
    trip_log = realloc(trip_log, want); // sole owner overwritten
    return trip_log ? 0 : -1;
}
```

▸ LeakSanitizer: detected memory leaks

What goes wrong: For an existing allocation and a nonzero request, allocation failure preserves the old object. Overwriting the sole owner with NULL then loses it. C11 zero-size allocation rules are separate; N3096 C23 makes zero-size realloc undefined in its nonnull-pointer branch. This helper rejects every zero request by policy.

✓ FIX

```
static char *trip_log;  // owned handle; initially NULL
int grow_log(size_t want) {
    if (want == 0) return -1;
    char *tmp = realloc(trip_log, want);
    if (tmp == NULL) return -1; // owner is unchanged
    trip_log = tmp;
    return 0;
}
```

Avoid it: Retain the owner until positive-size allocation succeeds. Validate count/element-size multiplication before requesting bytes; POSIX2024 specifies reallocarray for that task. allocator_may_return_null permits a supported sanitizer allocator to return failure instead of terminating; it does not force every request to fail.

Sources: C11 draft 7.22.3p1;7.22.3.5p2-p4;6.2.5p9; C23 draft N3096 7.24.3.7p3 nonnull-pointer zero-size branch; POSIX2024 DESCRIPTION:ERRORS; Sanitizer runtime docs allocator_may_return_null:detect_leaks; Memory host example capture C-MEM-09 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-09 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-10 sizeof on a pointer, not the array

SR-404 · Sensor sample buffer

✕ BUGGY

```
void zero_samples(int samples[16]) {
    memset(samples, 0, sizeof samples); // parameter is a
pointer
}
```

▸ gcc -Wsizeof-array-argument: 'sizeof' on array function parameter 's…

What goes wrong: The ordinary array parameter is adjusted to int*, so sizeof samples reports a pointer size. The exact cleared-byte count is a target property. A driver that leaves most samples initially zero cannot by itself demonstrate that the whole array was cleared.

> FIX

```
bool zero_samples(int *samples, size_t n) {
    if (n > SIZE_MAX / sizeof *samples) return false;
    if (n != 0 && samples == NULL) return false;
    for (size_t i = 0; i < n; ++i) samples[i] = 0;
    return true;
}
#define ARRAY_LEN(a) (sizeof (a) / sizeof (a)[0]) // actual
arrays only
/* int s[16]; zero_samples(s, ARRAY_LEN(s)); */
```

Sources: C11 draft 6.7.6.3p7;6.5.3.4;6.2.5p9;6.2.6.2p5;7.22.3.2 footnote296;7.24.6.1; GCC14 docs Wsizeof-array-argument;Wsizeof-pointer-memaccess; Memory host example capture C-MEM-10 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-10 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: The caller still supplies storage for n int elements; these checks cannot discover an arbitrary allocation capacity. The typed loop assigns integer zero and permits an empty NULL range. ARRAY_LEN is not a pointer-rejecting type check. Byte-count products can wrap, and a zero byte pattern is not a semantic zero for every replacement type. GCC14 documents selected sizeof diagnostics.

C-MEM-11 Stale pointer after realloc moved the block

SR-404 · Fault-code list

✕ BUGGY

```
int *v = malloc(4 * sizeof *v);
if (!v) return 1;
int *first = &v[0];               // alias into the block
int *tmp = realloc(v, 4096 * sizeof *v);
if (!tmp) { free(v); return 1; }
v = tmp;  // successful resize replaces the old object
*first = 1;  // old alias is invalid after success
free(v);
```

▸ gcc -Wuse-after-free=: pointer 'first' may be used after 'realloc'

What goes wrong: Successful realloc replaces the old object, even when the returned numeric address is the same. Old aliases cannot be evaluated as continuing pointers to it. Positive-size allocation failure is different: it leaves the old object allocated.

> FIX

```
int *v = NULL;
size_t count = 4, grown = 4096, first = 0;
if (count > SIZE_MAX / sizeof *v || grown == 0 ||
    grown > SIZE_MAX / sizeof *v) return 1;
v = malloc(count * sizeof *v);
if (v == NULL) return 1;
int *tmp = realloc(v, grown * sizeof *v);
if (tmp == NULL) { free(v); return 1; }
v = tmp;
if (first >= grown) { free(v); return 1; }
v[first] = 1; // derive from the returned live base
free(v);
```

Sources: C11 draft 6.2.4p2;7.22.3.5p2-p4;6.5.6; Clang docs Introduction;Limitations; C++17 draft N4659 26.3.11.3 [vector.capacity]p6,p10; Memory host example capture C-MEM-11 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-11 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Keep an index/offset when suitable, validate it against the new extent, then derive a pointer from the returned base. Do not compare an old alias after success to test whether it moved. ASan captures selected invalid accesses, not complete lifetime provenance. C++17 vector reallocation has a specific invalidation contract; other container operations need their own rules.

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/` . Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Strings & I/O

Station names on the CB radio, fault codes on the dash, the trip log on disk: text and I/O are where C quietly overflows or loses data.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 12 entries

C-STR-01 strcpy into a fixed buffer

SR-404 · Radio / CB station name (Ep 7)

✕ BUGGY

```
char station[8];
void set_station(const char *name) {
    strcpy(station, name);   // "KRAD 101.5" needs 11 bytes
}
```

▸ ASan: global-buffer-overflow

What goes wrong: strcpy copies through the terminating null and takes no destination capacity. Exceeding the destination violates its bounds; C does not promise neighboring-global corruption or a particular radio effect.

> FIX

```
char station[8];
int set_station(const char *name) {
    if (name == NULL) return -1;
    size_t n = strlen(name);
    if (n >= sizeof station) return -1;
    memmove(station, name, n + 1); // overlap allowed, includes null
    return 0;
}
```

Sources: C11 draft 7.24.2.2-7.24.2.3;7.21.6.5; glibc manual Levels1-3, unknown sizes and SIGABRT failure; Host example capture C-STR-01 exact supplied-example diagnostics/results; Defined host checks C-STR-01 defined inputs/error policy; host_abi observations

Avoid it: For this reject-without-changing policy, validate length before writing. The input must be a valid terminated string; memmove permits overlap. snprintf can modify/truncate before returning its would-have-written count. glibc fortification checks selected calls when size information is available; it does not make every overflow safe.

C-STR-02 strncpy leaves no terminator

SR-404 · Dashboard: fault code (DTC)

✕ BUGGY

```
char code[4];
strncpy(code, "P0301", sizeof code);  // copies "P030", no '\0'
printf("DTC %s\n", code);  // not a valid terminated string for %s
```

▸ gcc -Wstringop-truncation: 'strncpy' output truncated copying 4 byte…

What goes wrong: strncpy copies up to n characters. If no null occurs among those n characters it supplies no terminator; a shorter source is padded to n bytes, not necessarily the whole destination allocation.

> FIX

```
char code[6];  // five characters plus the terminating null
int n = snprintf(code, sizeof code, "%s", "P0301");
if (n < 0 || (size_t)n >= sizeof code) return 1;
printf("DTC %s\n", code);
```

Avoid it: Choose string formatting or fixed-width field copying deliberately. An n-1 plus explicit-null idiom requires positive capacity and valid storage; it may truncate. GCC14 documents -Wstringop-truncation. ASan reports selected executed invalid accesses, not every possible over-read.

Sources: C11 draft 7.24.2.4;7.21.6.5; GCC14 docs Wstringop-truncation; Clang docs Introduction;Limitations; Host example capture C-STR-02 exact supplied-example diagnostics/results

C-STR-03 Missing null terminator

SR-404 · SR-404 badge text

✕ BUGGY

```
char badge[3] = "404";   // legal C: the '\0' is silently dropped
puts(badge);             // requires a terminated string
```

▸ ASan: stack-buffer-overflow

What goes wrong: C11 permits a character array to hold exactly the non-null characters of a string literal. That array is not a terminated C string; C++17 disallows the shown initializer. Raw binary input needs a terminator only when later used with C-string operations.

> FIX

```
char badge[] = "404";    // compiler sizes it: 4 bytes incl. '\0'
puts(badge);
```

Avoid it: For this literal, infer the array length with []. For received text, check a successful nonnegative count and actual array capacity before adding a null. A pointer sizeof is not buffer capacity. GCC15 documents -Wunterminated-string-initialization in -Wextra.

Sources: C11 draft 6.7.9p14;7.1.1p1; C++17 draft N4659 [dcl.init.string]p2; GCC15 docs Wunterminated-string-initialization; Host example capture C-STR-03 exact supplied-example diagnostics/results

C-STR-04 Format string from data

SR-404 · Radio / CB messages

✕ BUGGY

```
void show_cb(const char *msg) {
    printf(msg);  // data must not become conversion instructions
}
```

▸ gcc -Wformat-security: format not a string literal and no format arg…

What goes wrong: A format string controls conversions and required arguments. Passing untrusted text as the format can violate that contract; missing or incorrectly typed arguments are undefined. Neither a particular memory leak nor remote code execution is guaranteed by this example.

> FIX

```
int show_cb(const char *msg) {
    return fputs(msg, stdout) == EOF ? -1 : 0;
}
```

Avoid it: Pass message text as data, as in fputs or printf("%s", msg), with a valid terminated string. Requiring literal formats is project guidance. GCC14 -Wformat=2 requests additional nonliteral/security checks; GCC format(printf,1,2) suits a wrapper with format first and checked arguments starting second.

Sources: C11 draft 7.21.6.1p8-p9;7.21.7.4; GCC14 docs Wformat=2;Wformat-security;Wformat-nonliteral; GCC14 docs format(archetype,string-index,first-to-check); Host example capture C-STR-04 exact supplied-example diagnostics/results; Defined host checks C-STR-04 defined inputs/error policy; host_abi observations

C-STR-05 Mismatched printf specifiers

SR-404 · Steering readout (Ep 2)

✕ BUGGY

```
double angle = 12.5;
size_t n = 3;
printf("%d deg, %d samples\n", angle, n);  // UB: wrong types
```

▸ gcc -Wformat=: format '%d' expects argument of type 'int', but argum…

What goes wrong: The format specifies required argument types after the applicable promotions. A double supplied for %d violates the contract. ISO C gives no portable garbage value, register/stack retrieval pattern or resulting steering motion.

> FIX

```
double angle = 12.5;
size_t n = 3;
printf("%.1f deg, %zu samples\n", angle, n);
```

Avoid it: Match actual argument types: %zu for size_t, %lld for long long, and the corresponding inttypes macros where those typedefs exist. GCC14 -Wformat checks documented cases; treating warnings as errors is a build policy. This mismatch example is compiled for a diagnostic, not executed.

Sources: C11 draft 6.5.2.2;7.21.6.1p7-p9;7.8.1; GCC14 docs Wformat; Host example capture C-STR-05 exact supplied-example diagnostics/results

C-STR-06 gets()

SR-404 · Diagnostics console

✕ BUGGY
```
char line[32];
gets(line);                 // no length limit: removed in
C11
```

▸ gcc error: implicit declaration of function 'gets'; did you mean 'fg…

What goes wrong: gets has no capacity parameter. C99TC3 deprecated it and C11 removed it from the standard library. GCC14 defaults make undeclared-function calls errors; library extensions and compiler options are separate from the language removal.

> FIX
```
char line[32];
if (fgets(line, sizeof line, stdin) == NULL)
    return ferror(stdin) ? 1 : 0;  // error or clean empty
EOF
char *nl = strchr(line, '\n');
if (nl != NULL) *nl = '\0';
else {
    int next = getchar();
    if (next != '\n' && next != EOF) return 1; // too long
    if (next == EOF && ferror(stdin)) return 1;
}
puts(line);
```

Sources: C11 draft Foreword: gets removed;7.21.7.2;7.21.7.6;7.21.10.3;7.24.5.2; C99TC3 draft N1256 7.19.7.7;7.26.9; GCC14 docs Implicit function declarations; Host example capture C-STR-06 exact supplied-example diagnostics/results; Defined host checks C-STR-06 defined inputs/error policy; host_abi observations

Avoid it: This one-command reader accepts NUL-free text that fits 31 characters, strips a newline and rejects a longer line. It checks EOF/error before using the buffer. A repeated-command reader also needs a resynchronization policy after rejection; selecting C17 alone does not prohibit every libc extension.

C-STR-07 scanf without width or result check

SR-404 · Throttle input (Ep 3)

✕ BUGGY
```
int main(void) {
    char name[16]; int pedal;
    scanf("%s %d", name, &pedal); // no width or result
check
    printf("%s %d\n", name, pedal);
    return 0;
}
```

▸ ASan: stack-buffer-overflow

What goes wrong: An unbounded %s can exceed the destination for a long token. A failed %d conversion does not assign the initially uninitialized pedal, and the nonmatching character can remain unread. A numeric result outside the destination range is undefined even if you check the conversion count afterward.

> FIX
```
int parse_pedal(const char *line, char name[16], int
*pedal) {
    size_t n = strcspn(line, " \t\r\n\v\f");
    if (n == 0 || n >= 16 || line[n] == '\0') return -1;
    const char *p = line + n;
    char *end;
    errno = 0;
    long v = strtol(p, &end, 10);
    if (errno || end == p || v < INT_MIN || v > INT_MAX)
        return -1;
    while (isspace((unsigned char)*end)) ++end;
    if (*end != '\0') return -1;
    memcpy(name, line, n);
    name[n] = '\0';
    *pedal = (int)v;
    return 0;
}
```

Sources: C11 draft 7.21.6.2p10,p12,p16;7.21.7.2;7.22.1.4;7.4;7.24.2.1;7.24.5.3; Clang docs Alias declaration; Clang docs Selected functions and strtol-family exclusion; Host example capture C-STR-07 exact supplied-example diagnostics/results; Defined host checks C-STR-07 defined inputs/error policy; host_abi observations

Avoid it: The fixed parser takes a terminated text line, a name buffer of at least 16 bytes and a valid output pointer. Its policy is one leading word followed by one representable int, with trailing whitespace allowed. The driver accepts NUL-free text and rejects overlong lines. Current cert-err34-c aliases a selected-call check that explicitly excludes strtol-family calls; validate those calls yourself.

C-STR-08 Unchecked fopen / fprintf / fclose

SR-404 · Trip computer log

✕ BUGGY
```
void save_trip(const char *path, long odo) {
    FILE *f = fopen(path, "w");     // may be NULL
    fprintf(f, "odo=%ld\n", odo);  // NULL violates the
FILE argument contract
    fclose(f);                      // flush errors ignored
}
```

▸ UBSan: null pointer passed as argument 1, which is declared to never…

What goes wrong: fopen can fail, and buffered writes can report failure at writing, flushing or closing. Ignoring failures can hide data loss; ignoring a successful return does not itself lose data. Successful library calls alone do not prove durable storage.

> FIX
```
int save_trip(const char *path, long odo) {
    FILE *f = fopen(path, "w");
    if (f == NULL) return -1;
    int ok = fprintf(f, "odo=%ld\n", odo) >= 0;
    if (fclose(f) != 0) ok = 0; // stream is closed even on
error
    return ok ? 0 : -1;
}
```

Avoid it: Check relevant open/write/close results and keep an error policy. A failed fclose still invalidates the stream. The Linux /dev/full fixture fails writes with ENOSPC; not every operation on it fails. GCC warn_unused_result and analyzer checks have compiler/path scope; C23 nodiscard recommends a diagnostic rather than guaranteeing every warning.

Sources: C11 draft 7.21.3;7.21.5.1-7.21.5.3;7.21.6.1p14;7.21.4.1; POSIX2024 fflush return/errors; Linux source v6.6 drivers/char/mem.c write_full; C23 draft N3096 6.7.12.2p4-p5 recommended diagnostics; GCC14 docs warn_unused_result; GCC14 docs Wanalyzer-null-argument; Host example capture C-STR-08 exact supplied-example diagnostics/results

C-STR-09 Ignoring short writes

SR-404 · Serial byte stream

✕ BUGGY
```
int send_frame(int fd, const char *buf, size_t n) {
    return write(fd, buf, n) < 0 ? -1 : 0;  // ignores
short writes
}
```

▸ prints: rc=0 sent=65536 of 1048576

What goes wrong: POSIX write reports bytes transferred and can return a positive count smaller than requested. For this byte-stream helper, that is partial progress, not completion. fwrite instead returns complete item counts; datagram send has a different message-boundary contract.

> FIX
```
int write_all(int fd, const char *buf, size_t n) {
    while (n > 0) {
        size_t chunk = n > (size_t)SSIZE_MAX ?
(size_t)SSIZE_MAX : n;
        ssize_t w = write(fd, buf, chunk);
        if (w < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (w == 0) return -1;
        buf += w;
        n -= (size_t)w;
    }
    return 0;
}
```

Sources: POSIX2024 Partial bytes;SSIZE_MAX;EINTR;EAGAIN; C11 draft 7.21.8.2p3 fwrite item count; POSIX2024 Message boundaries;EMSGSIZE; POSIX2024 Read count and EINTR; POSIX2024 Driver pipe ends; POSIX2024 Driver start argument/return; POSIX2024 Driver thread termination and join return; Host example capture C-STR-09 exact supplied-example diagnostics/results; Defined host checks C-STR-09 defined inputs/error policy; host_abi observations

Avoid it: The fixed helper is for valid buffers and blocking byte streams. It retries EINTR and rejects zero progress; other errors are returned. Nonblocking I/O needs a readiness/cancellation policy. Do not apply this loop blindly to datagrams or fwrite items. Test partial progress and error paths separately.

C-STR-10 atoi() with no error checking

SR-404 · Config: speed limit

✕ BUGGY

```c
int parse_limit(const char *arg) {
    return atoi(arg);  // no error report; an
unrepresentable result is UB
}
```

▸ prints: limit=0

What goes wrong: atoi cannot distinguish conversion errors through its return. No-conversion text produces 0, but a valid numeric prefix before trailing junk can produce a nonzero value. A result unrepresentable as int is undefined; a particular speed-limit effect requires the application policy.

> FIX

```c
int parse_limit(const char *arg, int *out) {
    if (arg == NULL || out == NULL) return -1;
    char *end;
    errno = 0;
    long v = strtol(arg, &end, 10);
    if (errno || end == arg || *end != '\0' || v < 0 || v >
255)
        return -1;
    *out = (int)v;
    return 0;
}
```

Sources: C11 draft 7.22.1p1;7.22.1.2;7.22.1.4; Clang docs Alias declaration; Clang docs Selected calls;strtol-family excluded; Host example capture C-STR-10 exact supplied-example diagnostics/results; Defined host checks C-STR-10 defined inputs/error policy; host_abi observations

Avoid it: The example policy accepts values 0..255, rejects trailing characters and leaves the output unchanged on failure. Check errno, end pointer and range before narrowing. Current cert-err34-c is an alias for a selected conversion-call check; its exclusion of strtol does not certify your error handling.

C-STR-11 getchar() stored in a char

SR-404 · Serial console

✕ BUGGY

```c
char c;                          // narrowing loses the int
result
int n = 0;
while ((c = getchar()) != EOF)  // captured 8-bit signed-
char/EOF=-1 target
    n++;
printf("%d bytes\n", n);
```

▸ prints: 2 bytes

What goes wrong: getchar returns int, including the negative EOF constant. C11 does not require EOF=-1 or 8-bit bytes. On this captured 8-bit signed-char host, narrowing 0xFF loses the distinction used by the loop. Other representations/conversion choices need their own analysis.

> FIX

```c
int c;  // retain the library result for the EOF comparison
size_t n = 0;
while ((c = getchar()) != EOF) {
    if (n == SIZE_MAX) return 1;
    ++n;
}
if (ferror(stdin)) return 1;
printf("%zu bytes\n", n);
```

Avoid it: Test EOF before narrowing an input character and distinguish read errors when needed. Plain-char signedness is implementation-defined. Character classifiers require EOF or a value representable as unsigned char; after excluding EOF, convert an actual character before isdigit/isalpha. Positive out-of-domain values are invalid too.

Sources: C11 draft 5.2.4.2.1;6.2.5p15;6.3.1.3;7.21.1p3;7.21.7.1;7.21.7.6;7.21.10.3;7.4p1; Host example capture C-STR-11 exact supplied-example diagnostics/results; Defined host checks C-STR-11 defined inputs/error policy; host_abi observations

C-STR-12 Comparing strings with ==

SR-404 · Brake command parser

✕ BUGGY

```c
bool is_brake_cmd(const char *cmd) {
    return cmd == "BRAKE";     // compares addresses, not
text
}
```

▸ gcc -Waddress: comparison with string literal results in unspecified…

What goes wrong: Pointer equality follows pointer-value rules, not string-content rules. Literal storage may be shared, so a coincidental equal pointer does not validate a text comparison algorithm.

> FIX

```c
bool is_brake_cmd(const char *cmd) {
    return cmd != NULL && strcmp(cmd, "BRAKE") == 0;
}
```

Avoid it: For valid terminated strings use a content comparison; use explicit lengths for other data. strncmp can establish a matching prefix without proving whole-string equality. GCC14 documents -Waddress under -Wall for selected literal-address comparisons; a warning is not complete validation.

Sources: C11 draft 6.4.5p7;6.5.9p6;7.24.4.1;7.24.4.2;7.24.4.4; GCC14 docs Waddress;Wall; Host example capture C-STR-12 exact supplied-example diagnostics/results; Defined host checks C-STR-12 defined inputs/error policy; host_abi observations

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Integers, Floats & Undefined Behaviour

The throttle, steering and GPS maths all ride on C's arithmetic rules: promotions, truncation, wraparound and the optimiser's assumptions about UB.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 13 entries

C-INT-01 Signed integer overflow

SR-404 · Odometer

✕ BUGGY
```c
int odo_add(int odo, int miles) {
    return odo + miles;  // core C overflow rule; compiler flags matter
}
```

▸ UBSan: signed integer overflow: 2147483647 + 1 cannot be represented…

What goes wrong: C11 core signed overflow is undefined. Optimizers can use the no-overflow premise; a particular folded comparison or deleted check depends on the compiler and options.

^ FIX
```c
bool odo_add(int odo, int miles, int *out) {
    if ((miles > 0 && odo > INT_MAX - miles) ||
        (miles < 0 && odo < INT_MIN - miles))
        return false;          // would overflow: report it
    *out = odo + miles;
    return true;
}
```

Sources: C11 draft 6.5p5; GCC14 docs Checked arithmetic builtins; Clang docs Checked Arithmetic Builtins; C23 draft N3096 7.20.1; GCC14 docs ftrapv; fwrapv; Host example capture C-INT-01 exact supplied-example diagnostics/results

Avoid it: Check representability before adding, or use a supported checked-arithmetic facility. GCC/Clang provide overflow builtins; C23 provides `ckd_add` from `<stdckdint.h>`. GCC14 `-ftrapv` / `-fwrapv` alter addition, subtraction and multiplication handling, not every operation.

C-INT-02 Integer promotion: narrow unsigned may become signed

SR-404 · Wheel-speed sensor

✕ BUGGY
```c
uint16_t raw = 65535;
uint32_t sq = raw * raw;  // on the recorded target, both promote to int:
                          // product exceeds INT_MAX → UB
printf("%" PRIu32 "\n", sq);
```

▸ UBSan: signed integer overflow: 65535 * 65535 cannot be represented …

What goes wrong: Integer promotions happen before arithmetic. With the recorded 32-bit int, uint16_t values promote to int and 65535 squared is unrepresentable. The complement result -16 additionally assumes the shown two's-complement representation.

^ FIX
```c
uint16_t raw = 65535;
uint32_t sq = (uint32_t)raw * raw;  // unsigned 32-bit on the recorded target
printf("%" PRIu32 "\n", sq);
```

Avoid it: Choose an operation type with sufficient range after promotions, and convert before multiplying. Shifts also require a valid count. Requested conversion warnings and sanitizer checks are not complete proofs.

Sources: C11 draft 6.3.1.1;6.3.1.8;6.2.6.2;6.5.3.3; GCC14 docs signed-integer-overflow; GCC14 docs Wconversion; Wsign-conversion; Host example capture C-INT-02 exact supplied-example diagnostics/results; Defined host checks host_abi: actual widths/radix/double comparison; not full AnnexF conformance

C-INT-03 Unsigned wraparound in a countdown loop

SR-404 · Radio text scroller

✕ BUGGY
```c
void print_rev(const char *s) {
    for (size_t i = strlen(s) - 1; i >= 0; i--)  // always true
        putchar(s[i]);  // "" → i = SIZE_MAX
}
```

▸ gcc -Wtype-limits: comparison of unsigned expression in '>= 0' is al…

What goes wrong: An unsigned value cannot be negative, so i >= 0 cannot end this loop. Converting -1 to size_t yields SIZE_MAX. The type of strlen(s) - 1 itself depends on integer promotions.

^ FIX
```c
void print_rev(const char *s) {
    for (size_t i = strlen(s); i-- > 0; )  // test, then decrement
        putchar(s[i]);
}
```

Avoid it: The shown i-- > 0 loop avoids an empty-string access, but i can wrap after the final test. Rewriting a - b < c as a < b + c requires a >= b and representable b + c for equivalence.

Sources: C11 draft 6.2.5p9;6.3.1.1;6.3.1.8;6.5.2.4;7.19; GCC14 docs Wtype-limits; Wextra; Host example capture C-INT-03 exact supplied-example diagnostics/results

C-INT-04 Shift too far, or into the sign bit

SR-404 · Lamp bitmask

✕ BUGGY
```c
uint32_t bit_mask(int bit) {
    return 1 << bit;  // 1 is int; count/value constraints depend on promoted int
}
```

▸ UBSan: shift exponent 32 is too large for 32-bit type 'int'

What goes wrong: A negative count or a count at least the promoted left operand's width is undefined in C11. Signed left shift also has value constraints. Negative signed right shift is implementation-defined; no particular C result follows from hardware count masking.

^ FIX
```c
uint32_t bit_mask(unsigned bit) {
    return bit < 32 ? UINT32_C(1) << bit : 0;  // range-checked
}
```

Avoid it: Check the promoted operation type, value range and count. UINT32_C corresponds to promoted uint_least32_t, not necessarily an exact 32-bit unsigned operation. GCC14 and Clang document shift sanitizers; GCC extensions are separate from ISO rules.

Sources: C11 draft 6.5.7;7.20.4;7.20.4.1; GCC14 docs Signed shift extension and diagnosis; Clang docs shift checks; Host example capture C-INT-04 exact supplied-example diagnostics/results

C-INT-05 Division by zero (and INT_MIN / -1)

SR-404 · Average RPM

✕ BUGGY
```c
int avg_rpm(int total, int count) {
    return total / count;  // zero divisor: core C undefined behavior
}
```

▸ UBSan: division by zero

What goes wrong: Integer division needs a nonzero divisor and a representable quotient. INT_MIN / -1 is invalid on an asymmetric signed range. A required process signal is not an ISO C guarantee. Floating zero-divisor behavior depends on the language/IEC60559 contract and environment.

^ FIX
```c
bool avg_rpm(int total, int count, int *out) {
    if (count == 0 || (total == INT_MIN && count == -1))
        return false;  // reject zero; conservatively reject INT_MIN / -1
    *out = total / count;
    return true;
}
```

Avoid it: Check external divisors and quotient range before dividing. Use documented executed-case checks, not a promised outcome from undefined behavior. isfinite detects nonfinite results when the floating environment permits returning them.

Sources: C11 draft 6.5.5p5-p6;Annex F.1/F.3/F.7;7.12.3.2; GCC14 docs integer-divide-by-zero; Host example capture C-INT-05 exact supplied-example diagnostics/results

C-INT-06 Integer division truncates

SR-404 · Throttle (Ep 3)

✕ BUGGY
```c
float throttle_cmd(int pedal_mm, int travel_mm) {
    return pedal_mm / travel_mm;   // 30 / 40 → 0, not 0.75
}
```

▸ prints: throttle=0.00

What goes wrong: Defined integer division discards the fraction before a later float conversion. The shown 30 / 40 becomes integer 0 before conversion, whereas floating division gives 0.75; this is an example, not a measured whole-car response.

^ FIX
```c
float throttle_cmd(int pedal_mm, int travel_mm) {
    if (travel_mm <= 0) return 0.0f;
    return (float)pedal_mm / (float)travel_mm;  // 30 / 40 → 0.75
}
```

Avoid it: Convert an operand before dividing and inspect integer intermediates. The named clang-tidy bugprone-integer-division check has documented exclusions. Test fractional inputs; a tool warning is not proof of all-input correctness.

Sources: C11 draft 6.5.5p6;6.3.1.4;6.3.1.8; Clang docs check description and exclusions; Host example capture C-INT-06 exact supplied-example diagnostics/results

C-INT-07 Strict aliasing (type punning via pointer cast)

SR-404 · CAN frame decoder

✕ BUGGY
```c
float f = 1.0f;
uint32_t bits = *(uint32_t *)&f;  // invalid access on the shown target
printf("%08" PRIx32 "\n", bits);
```

▸ gcc -Wstrict-aliasing: dereferencing type-punned pointer will break …

What goes wrong: C11 effective-type rules have specified exceptions. On the captured target, accessing this float through uint32_t * violates them. Optimization can expose the invalid access; alignment and object validity must also hold.

> FIX
```c
float f = 1.0f;
_Static_assert(sizeof(float) == sizeof(uint32_t), "32-bit float required");
uint32_t bits;
memcpy(&bits, &f, sizeof bits);  // copies representation; no instruction-count promise
printf("%08" PRIx32 "\n", bits);
```

Sources: C11 draft 6.5p7;6.3.2.3p7;6.2.6.1;7.24.2.1; C++20 draft N4861 [bit.cast] constraints; GCC14 docs Wstrict-aliasing; GCC14 docs fstrict-aliasing; Host example capture C-INT-07 exact supplied-example diagnostics/results

Avoid it: Use memcpy with size/representation checks; exact hex also needs byte-order assumptions. C++20 bit_cast requires equal size and trivially copyable types. GCC strict-aliasing diagnostics require active `-fstrict-aliasing`; a flag change does not repair unrelated errors.

C-INT-08 Unsequenced modifications (i++ + i++)

SR-404 · Sample buffer

✕ BUGGY
```c
int i = 1;
int x = i++ + i++;        // two unsequenced writes to i: UB
int buf[4] = {0};
buf[i] = i++;  // UB in C; C++17 uses the incremented index
printf("%d %d\n", x, buf[2]);
```

▸ gcc -Wsequence-point: operation on 'i' may be undefined

What goes wrong: i++ + i++ is unsequenced in C11 and C++17. Assignment sequencing differs: an isolated C++17 buf[i] = i++ uses the incremented index and can access buf[4] when i starts at 3. Earlier undefined behavior does not define a later whole-program trace.

> FIX
```c
int i = 1;
int x = i + (i + 1);      // what was meant, with no side effects
i += 2;
int buf[4] = {0};
buf[i] = i;               // index and value from the same i
i++;
printf("%d %d\n", x, buf[3]);
```

Avoid it: Separating updates and accesses is conservative style guidance, not a language mandate for every expression. GCC `-Wsequence-point` and Clang `-Wunsequenced` request diagnostics. Review macros that duplicate argument evaluation.

Sources: C11 draft 6.5p2;6.10.3.1; C++17 draft N4659 [intro.execution];[expr.ass]; GCC14 docs Wsequence-point; Clang docs Wunsequenced; Host example capture C-INT-08 exact supplied-example diagnostics/results

C-INT-09 Signed/unsigned comparison and silent narrowing

SR-404 · Steering damping (Ep 2)

✕ BUGGY
```c
int temp_c = -5;
unsigned limit = 100;
if (temp_c < limit)  // same-rank int/unsigned int conversion
    puts("in range");            // never printed
int16_t rpm = read_rpm();  // out-of-range signed conversion: implementation-defined
```

▸ gcc -Wsign-compare: comparison of integer expressions of different s…

What goes wrong: Mixed integer operands follow rank and representability rules; they do not always convert to unsigned. Out-of-range signed integer conversion is implementation-defined or can signal in C11. Floating truncation is a separate rule, not dropping high integer bits.

> FIX
```c
int temp_c = -5;
unsigned limit = 100;
if (temp_c >= 0 && (unsigned)temp_c < limit)
    puts("in range");
int raw = read_rpm();
int16_t rpm = (int16_t)(raw > INT16_MAX ? INT16_MAX
                        : raw < 0 ? 0 : raw);   // saturate
```

Avoid it: Validate the source value and destination range before casting. The shown RPM clamp is this exercise's chosen nonnegative range. GCC14 `-Wconversion`/`-Wsign-conversion` request selected implicit-conversion diagnostics; explicit casts are not validation.

Sources: C11 draft 6.3.1.3;6.3.1.4;6.3.1.8; GCC14 docs Wsign-compare;Wconversion;Wsign-conversion; Host example capture C-INT-09 exact supplied-example diagnostics/results

C-INT-10 Comparing floats with ==

SR-404 · Fuel gauge

✕ BUGGY
```c
double v = 0.1 + 0.2;
if (v == 0.3)  // captured binary64 target: comparison is false
    puts("equal");
```

▸ gcc -Wfloat-equal: comparing floating-point with '==' or '!=' is uns…

What goes wrong: These fractions are not exact finite binary fractions; the captured binary example compares unequal after rounding. ISO C need not use a binary radix. Under the IEC60559 quiet-NaN contract, a NaN never compares equal, even to itself.

> FIX
```c
double v = 0.1 + 0.2;
if (fabs(v - 0.3) <= 1e-9 * fmax(1.0, fabs(v)))  // tolerance
    puts("equal");
```

Avoid it: Choose equality or an error tolerance for the actual numerical contract. isnan detects NaNs; isinf detects infinities; isfinite rejects either nonfinite class. GCC14 `-Wfloat-equal` requests floating-equality diagnostics, not every == or a correctness proof.

Sources: C11 draft 5.2.4.2.2;Annex F.2.1/F.3;7.12.3.2-7.12.3.4; Goldberg/Oracle Binary representation and error; GCC14 docs Wfloat-equal; Host example capture C-INT-10 exact supplied-example diagnostics/results; Defined host checks host_abi; actual widths/radix/double comparison; not full AnnexF conformance

C-INT-11 Out-of-range float → int conversion

SR-404 · Steering rack

✕ BUGGY
```c
int steer_counts(double angle) {
    return (int)(angle * 1000.0);  // out-of-range conversion; see core/Annex F rules
}
```

▸ UBSan: 1e+10 is outside the range of representable values of type 'i…

What goes wrong: The C11 core out-of-range floating conversion rule differs from optional Annex F: Annex F raises invalid and gives an unspecified result. Neither contract guarantees a portable integer result or rack motion.

> FIX
```c
int steer_counts(double angle) {
    if (!isfinite(angle)) return 0;
    if (angle > 30.0) return 30000;
    if (angle < -30.0) return -30000;
    return (int)(angle * 1000.0); // bounded before scaling
}
```

Avoid it: Check finiteness and bound the input before scaling or converting. The fixed counts range is an exercise policy. GCC14 excludes float-cast-overflow from undefined; retrieved Clang documentation includes it. Name the compiler and requested check.

Sources: C11 draft 6.5p5;6.3.1.4;Annex F.1/F.4; GCC14 docs float-cast-overflow groups; Clang docs float-cast-overflow groups; Host example capture C-INT-11 exact supplied-example diagnostics/results

C-INT-12 Degrees passed to sin/cos

SR-404 · GPS / minimap side quest

✕ BUGGY
```c
double lon_scale(double lat_deg) {
    return cos(lat_deg);  // cos() takes radians: 37 rad, not 37°
}
```

▸ prints: 0.765

What goes wrong: sin, cos and tan take angle arguments in radians. atan2 takes y then x and returns the principal angle in radians for valid numeric inputs. A particular minimap distortion requires a caller/projection model.

> FIX
```c
static const double kPi = 3.141592653589793;  // M_PI isn't ISO C
double lon_scale(double lat_deg) {
    return cos(lat_deg * kPi / 180.0);   // 37° → 0.799
}
```

Avoid it: Make units explicit and convert at boundaries. Test known mathematical values with an appropriate floating error policy: cos 60 degrees is mathematically 0.5. M_PI is specified by POSIX/XSI, not required by ISO C11.

Sources: C11 draft 7.12.4.4-7.12.4.7;7.12; POSIX math M_PI, XSI option; Host example capture C-INT-12 exact supplied-example diagnostics/results

C-INT-13 % keeps the sign of the dividend

SR-404 · GPS turn arrow

✕ BUGGY

```
int wrap_heading(int deg) {
    return deg % 360;          // -350 % 360 == -350, not 10
}
```

▸ prints: -350

What goes wrong: For defined integer division, a nonzero remainder has the dividend's sign. fmod has related sign behavior for valid finite inputs, but different domains and exceptions. A negative remainder alone does not establish a nav-arrow motion.

> FIX

```
int wrap_heading(int deg) {
    int r = deg % 360;
    return r < 0 ? r + 360 : r;   // always 0..359
}
```

Avoid it: For positive m and a valid division, r < 0 ? r + m : r gives a nonnegative representative. Test negative and half-turn inputs. remainder(x,360.0) permits both +180 and -180 by its ties-to-even quotient rule; choose the intended endpoint policy.

Sources: C11 draft 6.5.5p6;7.12.10.1-7.12.10.2;footnote239; Host example capture C-INT-13 exact supplied-example diagnostics/results

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/` . Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Preprocessor & Everyday Slips

Macros, headers, switches and one-character typos: language constraints, diagnostic scope and explicit control-flow policies.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2 · -fanalyzer · -fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 13 entries

C-PP-01 Macro without parentheses

SR-404 · Turbo boost calc

BUGGY
```c
#define SQUARE(x) x * x
int boost(int rpm) {
    return SQUARE(rpm + 1);   // boost(3): expansion yields 7
}
```

▸ prints: boost=7

What goes wrong: Macro replacement operates on preprocessing tokens. The unparenthesized expansion is rpm + 1 * rpm + 1; boost(3) yields 7. Parentheses fix grouping but not repeated evaluation or signed overflow.

FIX
```c
#define SQUARE(x) ((x) * (x))   // if it must be a macro
static inline int square(int x) { return x * x; }  // typed; still requires representable product
int boost(int rpm) {
    return square(rpm + 1);   // 16
}
```

Avoid it: Author guidance: prefer a typed function when suitable. The shown square requires x*x to fit int, and boost also requires rpm+1 to fit. Parenthesize expression-macro operands and result; GCC14 documents -E for inspecting preprocessing. C const objects are not interchangeable with enum integer constant expressions.

Sources: C11 draft 6.10.3; 6.5p5; 6.6; GCC14 preprocessor options -E

C-PP-02 Macro evaluates an argument twice

SR-404 · Sensor FIFO

BUGGY
```c
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int next_sample(void);            // pops the sensor FIFO
int peak(int limit) {
    return MAX(next_sample(), limit);  // may pop the FIFO twice
}
```

▸ prints: peak=102 calls=2

What goes wrong: The conditional operator evaluates its condition and only the selected operand. Here a sample above limit causes a second call; a sample not above limit is consumed only once. Repeated tokens do not imply every occurrence executes. The supplied driver shows the two-call branch.

FIX
```c
static inline int max_int(int a, int b) { return a > b ? a : b; }
int peak(int limit) {
    return max_int(next_sample(), limit);  // evaluated exactly once
}
```

Avoid it: Author guidance: inspect an expression macro before passing side effects. A function evaluates each argument once, but C does not specify the order among arguments. Keep interdependent side effects in separate statements. C11 _Generic selects by type; its controlling expression is not evaluated.

Sources: C11 draft 6.10.3.1; 6.5.15; 6.5.2.2; 6.5.1.1

C-PP-03 Multi-statement macro without do/while(0)

SR-404 · Gauge cluster reset

BUGGY
```c
#define RESET(g) (g).speed = 0; (g).rpm = 0
void on_fault(struct gauge *g, bool fault) {
    if (fault)
        RESET(*g);          // only .speed is conditional
}
```

▸ prints: speed=50 rpm=0

What goes wrong: In this expansion only the speed assignment is the if body; the rpm assignment runs even when fault is false. A compound-block macro followed by the caller's semicolon can separate an else from its if. do/while(0) packages a statement-like macro.

FIX
```c
#define RESET(g) do { (g).speed = 0; (g).rpm = 0; } while (0)
void on_fault(struct gauge *g, bool fault) {
    if (fault)
        RESET(*g);          // one statement; needs its ';'
}
```

Avoid it: Author guidance: prefer braces and typed helpers when practical. The do/while wrapper does not prevent repeated evaluation of g: this macro still uses it twice. Pass a stable valid gauge lvalue; adding break/continue inside such a macro needs separate control-flow review.

Sources: C11 draft 6.8.4.1; 6.8.5.2; 6.10.3

C-PP-04 Header without include guard

SR-404 · Engine computer modules (Ep 10)

BUGGY
```c
/* gauge.h */
struct gauge { int value; };

/* dash.c */
#include "gauge.h"
#include "gauge.h"   // repeated inclusion in this translation unit
```

▸ gcc error: redefinition of 'struct gauge'

What goes wrong: This header repeats a complete definition of the same struct tag in one scope when included twice, violating C's type-definition constraint. Repeating a compatible declaration is not always invalid; it is this repeated definition that requires protection.

FIX
```c
/* gauge.h */
#ifndef KWH_GAUGE_H
#define KWH_GAUGE_H
struct gauge { int value; };
#endif /* KWH_GAUGE_H */
```

Avoid it: Author guidance: use a unique project-prefixed guard for a header meant for repeated inclusion. All identifiers beginning with underscore plus uppercase, or double underscore, are reserved for any use. The #ifndef/#define mechanism is specified by ISO C; implementation-specific alternatives need their own documentation.

Sources: C11 draft 6.7.2.3p1; 6.10.1; 7.1.3

C-PP-05 = instead of ==

SR-404 · Park interlock

BUGGY
```c
void check_park(int gear) {
    if (gear = 0)          // assigns 0: never true
        engage_park();
}
```

▸ gcc -Wparentheses: suggest parentheses around assignment used as tru…

What goes wrong: Assignment yields the value stored after conversion. Here assigning int zero makes the condition false and changes the local parameter. A nonzero value is true after conversion; a nonzero source constant need not remain nonzero for every destination type.

FIX
```c
void check_park(const int gear) {  // assignment to const violates a constraint
    if (gear == 0)
        engage_park();
}
```

Avoid it: GCC14 documents -Wparentheses in -Wall for selected assignment-in-condition cases. A const-qualified local parameter makes this assignment violate a modifiable-lvalue constraint. Author guidance: express deliberate assignment and comparison clearly; warnings are not proof of the intended condition.

Sources: C11 draft 6.5.16; 6.8.4.1; GCC14 docs -Wparentheses; -Wall

C-PP-06 Switch fall-through

SR-404 · Brakes / ABS modes (Ep 4)

✕ BUGGY

```
enum brake_mode { ABS_ON, ABS_OFF };
int pressure(enum brake_mode m) {
    int p = 0;
    switch (m) {
    case ABS_ON:  p = 80;     // missing break: falls into
ABS_OFF
    case ABS_OFF: p = 100; break;
    }
    return p;
}
```

▸ `gcc -Wimplicit-fallthrough=: this statement may fall through`

What goes wrong: After the switch jumps to ABS_ON, execution continues to the next statement and then the ABS_OFF label because no break/return intervenes. This function returns the integer100; pressure units and real ABS behavior are a teaching model, not a vehicle specification.

Sources: C11 draft 6.8.4.2; 6.8.6.3; GCC14 docs -Wimplicit-fallthrough; -Wswitch; -Wswitch-enum

✓ FIX

```
enum brake_mode { ABS_ON, ABS_OFF };
int pressure(enum brake_mode m) {
    switch (m) {
    case ABS_ON:  return 80;
    case ABS_OFF: return 100;
    }
    return 0;   // GCC -Wswitch can warn about omitted
enumerators
}
```

Avoid it: GCC14 documents -Wimplicit-fallthrough=3 in -Wextra and -Wswitch in -Wall. A default label suppresses missing-enumerator warnings from -Wswitch; -Wswitch-enum can warn even with default. Author guidance: use explicit returns/breaks; the GNU fallthrough attribute can mark a deliberate fall-through.

C-PP-07 Signal flag without volatile sig_atomic_t

SR-404 · Ignition-off handler

✕ BUGGY

```
static int stop; // invalid asynchronous flag object
static void on_alarm(int sig) { (void)sig; stop = 1; }
void run_loop(void) {
    signal(SIGALRM, on_alarm);
    alarm(1);
    while (!stop) { }  // invalid async sharing; no
required C outcome
}
```

▸ `hangs: killed by the test watchdog`

What goes wrong: The asynchronous handler's assignment to this plain static int violates C11's signal-handler object restrictions. A timeout is one bounded host observation, not a required optimized outcome or a bug restricted to optimized builds. The POSIX SIGALRM/alarm fixture is not portable ISO C.

✓ FIX

```
static volatile sig_atomic_t stop;
static void on_alarm(int sig) { (void)sig; stop = 1; }
int run_loop(void) { // POSIX, single invocation on this
host
    if (signal(SIGALRM, on_alarm) == SIG_ERR) return -1;
    (void)alarm(1);
    while (!stop) { } // volatile flag; delivery time is
not guaranteed
    return 0;
}
```

Avoid it: For a simple asynchronous flag, C11 permits assigning a volatile sig_atomic_t; other handler actions have separate restrictions. The fixed example checks signal installation and waits for actual delivery, without a one-second scheduling guarantee. Volatile does not provide general thread synchronization or a portable memory-mapped-device protocol.

Sources: C11 draft 7.14; 7.14.1.1p5; 6.7.3; POSIX alarm DESCRIPTION; RETURN VALUE; APPLICATION USAGE; POSIX signal rules 2.4 Signals; POSIX signal DESCRIPTION; RETURN VALUE; APPLICATION USAGE

C-PP-08 Missing #include (implicit declaration)

SR-404 · Dashboard hello.c (Ep 1)

✕ BUGGY

```
int main(void) {
    printf("Hello, World!\n");   // no #include <stdio.h>
    return 0;
}
```

▸ `gcc error: implicit declaration of function 'printf'`

What goes wrong: C11 requires a declared function designator and a diagnostic for a constraint violation; it does not prescribe GCC's warning severity or require translation to stop. GCC14 documents an error by default for implicit function declarations in C99 and later modes, with options that can downgrade it.

✓ FIX

```
#include <stdio.h>
int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Avoid it: Author guidance: include the header declaring each library function, here stdio.h for printf, instead of guessing a prototype. Select the language mode and warning policy explicitly. GCC's -Werror=implicit-function-declaration requests an error for that diagnostic; it is a compiler option, not an ISO C rule.

Sources: C11 draft 5.1.1.3; 6.5.1; 7.21.6.3; GCC14 docs -Wimplicit-function-declaration; GCC14 docs Implicit function declarations

C-PP-09 Missing semicolon

SR-404 · Dashboard (Ep 1)

✕ BUGGY

```
int main(void) {
    printf("Hello, World!\n")    // missing ;
    return 0;  // shown GCC diagnostic is near return
}
```

▸ `gcc error: expected ';' before 'return'`

What goes wrong: A C expression statement ends with a semicolon. Here the missing semicolon makes the source syntactically invalid. Diagnostic wording and location depend on the implementation; a later-line diagnostic can be a clue to an earlier missing token, not a universal parser rule.

Sources: C11 draft 5.1.1.3; 6.8.3; 6.7

✓ FIX

```
int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Avoid it: Author guidance: start with the first diagnostic and inspect nearby preceding syntax. Struct declarations also need their terminating semicolon. Formatting can help inspection but does not prove syntax or intended behavior. The shown supplied example is checked separately from the game's runtime path.

C-PP-10 #if on an undefined macro

SR-404 · Debug trace build

✕ BUGGY

```
#if KWH_DEBUG_LVL > 1          // undefined here: replaced
with 0
#define TRACE(msg) fputs(msg, stderr)
#else
#define TRACE(msg) ((void)0)
#endif
```

▸ `gcc -Wundef: 'KWH_DEBUG_LVL' is not defined, evaluates to '0'`

What goes wrong: After macro replacement and defined processing in a #if expression, remaining identifiers become preprocessing-number0. With KWH_DEBUG_LVL absent this selects the disabled trace branch. GCC can warn about the undefined identifier; silence is not guaranteed for every build.

Sources: C11 draft 6.10.1p4; GCC14 preprocessor options -Wundef; -dM; -E

✓ FIX

```
#ifndef KWH_DEBUG_LVL
#define KWH_DEBUG_LVL 0         // explicit, documented
default
#endif
#if KWH_DEBUG_LVL > 1           // now a deliberate choice
/* ... TRACE definitions as before ... */
```

Avoid it: GCC14 documents -Wundef and -dM with -E. Author guidance: choose either a numeric-level policy or a defined/absent policy. #ifdef tests existence, so a macro defined as0 still passes it. Defaults can be intentional, but required configuration can instead be rejected with #error.

C-PP-11 Misleading indentation / missing braces

SR-404 · Bus frame handler

✕ BUGGY

```
void on_frame(int crc_ok) {
    if (crc_ok)
        apply_frame();
        log_frame();     // looks guarded, always runs
}
```

▸ `gcc -Wmisleading-indentation: this 'if' clause does not guard...`

What goes wrong: An unbraced if controls its single following statement, regardless of indentation. For crc_ok==0 the supplied bad function still calls log_frame. This is a language-rule demonstration; no historical security incident is required to establish it.

> FIX

```
void on_frame(int crc_ok) {
    if (crc_ok) {
        apply_frame();
        log_frame();
    }
}
```

Avoid it: Author guidance: brace conditional bodies and format consistently. GCC14 documents -Wmisleading-indentation in -Wall for selected misleading layouts, with layout/preprocessor limitations. A clean diagnostic run does not establish that every intended statement is guarded.

Sources: C11 draft 6.8.4.1; GCC14 docs -Wmisleading-indentation

C-PP-12 Inverted or mis-negated condition

SR-404 · Cargo top-box lock (side quest)

✕ BUGGY

```
int topbox_lock_cmd(int key, int unlock_req, int
latch_closed) {
    if (key && unlock_req && latch_closed == 0)  // needs
it open!
        return 1;                                // 1 =
unlock
    return 0;
}
```

▸ `prints: cmd=0`

What goes wrong: For the stated policy, unlocking requires key, request and a closed latch. The bad comparison rejects that all-true input. De Morgan's Boolean identity is !(a&&b) == (!a||!b), but arbitrary C rewrites also need review of side effects and evaluation order. The cargo story is a game model.

> FIX

```
int topbox_lock_cmd(bool key, bool unlock_req, bool
latch_closed) {
    if (key && unlock_req && latch_closed)       // closed
+ request
        return 1;
    return 0;
}
```

Avoid it: Author guidance: use clear predicates and check all eight Boolean combinations against an explicit policy. bool parameters normalize scalar values; they do not validate a real sensor or authorization. Static analysis can detect selected logic issues, but diagnostic scope depends on tool and specification.

Sources: C11 draft 6.3.1.2; 6.5.3.3; 6.5.13; 6.5.14

C-PP-13 Global variable defined in a header

SR-404 · Engine computer modules (Ep 10)

✕ BUGGY

```
/* speed.h */
int speed_kph;  // tentative definition; also a declaration

/* dash.c and cruise.c both #include "speed.h" */
```

▸ `ld: multiple definition of `speed_kph'`

What goes wrong: At file scope int speed_kph is a tentative definition, and also a declaration. In each translation unit without another definition it behaves as a zero-initialized definition. Placing it in both units violates the external-definition requirement. GCC10's porting guide documents the default change to -fno-common; -fcommon is an extension workaround.

> FIX

```
/* speed.h */
extern int speed_kph;          // declaration only

/* speed.c: exactly one definition */
#include "speed.h"
int speed_kph = 0;
```

Avoid it: Author guidance: place an extern declaration in the header and one external definition in a source file. An extern declaration with an initializer would itself be a definition. File-scope static gives internal linkage and separate objects per translation unit when placed in a header; it is not a shared-global repair.

Sources: C11 draft 6.9p5; 6.9.2; 6.2.2; GCC10 porting guide Default to -fno-common

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.

C++ Objects & Ownership

Copy/move policy, borrowed lifetimes, cleanup and invalidation, with explicit C++17 and host-check scope.

Debug build:
`-std=c++17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 11 entries

CPP-OBJ-01 Rule of 3/5 violation (shallow copy)

SR-404 · Climate-control buffer

× BUGGY
```
struct Buffer {
    char* data;
    explicit Buffer(std::size_t n) : data(new char[n]) {}
    ~Buffer() { delete[] data; }
};   // copy constructor/assignment are still the implicit ones
// Buffer a(16); Buffer b = a; // implicit copies alias one allocation
```

▸ ASan: attempting double-free

What goes wrong: In this Buffer, implicit copying copies the pointer value, not the allocated characters. Copying a successfully allocated buffer creates two handles whose destructors attempt to delete the same allocation. Raw pointers are not inherently owners; this example makes data an owner by its destructor.

> FIX
```
struct Buffer {                          // Rule of Zero
    std::vector<char> data;
    explicit Buffer(std::size_t n) : data(n) {}
};   // member operations implement this character-value policy
```

Avoid it: Author guidance: prefer resource-managing members when they express the intended value/ownership policy. Here vector gives independent character storage on copy. The Rule of Zero/Five is design guidance, not an ISO requirement to hand-write five functions; review defaulted, deleted and custom operations together.

Sources: C++17 draft 15.8.1 [class.copy.ctor]; 15.8.2 [class.copy.assign]; 26.3.11 [vector]; C++ Core Guidelines (guidance) C.20; C.21

CPP-OBJ-02 Object slicing

SR-404 · Sensor polling

× BUGGY
```
struct Sensor {
    virtual ~Sensor() = default;
    virtual int read() const { return 0; }
};
struct RpmSensor : Sensor {
    int read() const override { return 3000; }
};
int poll(Sensor s) { return s.read(); }  // slices: base part only
```

▸ prints: poll=0

What goes wrong: Constructing the by-value Sensor parameter from RpmSensor creates a separate Sensor object. Calling read on that parameter uses Sensor's implementation; it does not change the original derived object. Likewise, constructing vector<Sensor> elements from derived values cannot preserve their dynamic type.

> FIX
```
// Sensor and RpmSensor unchanged
int poll(const Sensor& s) { return s.read(); }  // dispatches
```

Avoid it: Author guidance: use references/pointers when dynamic dispatch to the original live object is intended. Owning unique_ptr elements are one storage policy, not the only one. Reference/pointer lifetimes still matter; the supplied temporary remains alive through the poll call.

Sources: C++17 draft 15.8.1 [class.copy.ctor]; 13.3 [class.virtual]; 15.2 [class.temporary]

CPP-OBJ-03 Raw new/delete leaks on exceptions

SR-404 · Fuel-map tuning

× BUGGY
```
void tune_fuel_map() {
    auto* map = new FuelMap();
    calibrate(*map);      // throws → delete never runs
    delete map;
}
```

▸ LeakSanitizer: detected memory leaks

What goes wrong: In the supplied fixture, calibrate throws after allocation and an outer handler catches it. The raw pointer has no cleanup operation on that unwinding path. With unique_ptr, destruction during this unwinding releases the object. Not every exception/early return between allocation and delete necessarily leaks.

> FIX
```
void tune_fuel_map() {
    auto map = std::make_unique<FuelMap>();
    calibrate(*map);  // outer catch unwinding releases the owner
}
```

Avoid it: Author guidance: express owned resources through RAII members or handles, with the correct deleter. The supplied catch is part of this cleanup demonstration; terminating paths do not universally promise stack unwinding. Clang documents platform/configuration limits for leak detection; the host receipt covers this particular run.

Sources: C++17 draft 18.2 [except.ctor]; 18.5.1 [except.terminate]; 23.11.1.2.2 [unique.ptr.single.dtor]; Clang docs Memory leak detection; Limitations; C++ Core Guidelines (guidance) R.11; R.20; E.6

CPP-OBJ-04 Iterator invalidation

SR-404 · Fault-code list

× BUGGY
```
std::vector<int> faults{1, 2, 3};
while (faults.size() < faults.capacity())
faults.push_back(0);
for (int f : faults)
    if (f == 2)
        faults.push_back(99);  // exceeds capacity: invalidates these iterators
```

▸ ASan: heap-use-after-free

What goes wrong: Range-for caches begin/end. N4659 preserves pre-insertion handles without reallocation; N4861 explicitly invalidates the old end even without reallocation. This bad fixture fills spare capacity first, so append requires reallocation and invalidates element iterators under both drafts. Erase/reserve have distinct rules.

> FIX
```
std::vector<int> faults{1, 2, 3};
const std::size_t n = faults.size();
for (std::size_t i = 0; i < n; ++i)  // original range; append only
    if (faults[i] == 2)
        faults.push_back(99);
```

Avoid it: Author guidance: this append-only fix visits the original n elements by index and keeps no element reference across append. It does not cover erasing/shrinking or every container. Reallocation invalidates element handles; reserve within current capacity does not. Sanitizer/debug checks have executed-path and library scope.

Sources: C++17 draft 9.5.4 [stmt.ranged]; 26.3.11.3 [vector.capacity]; 26.3.11.5 [vector.modifiers]; Clang docs Introduction; Limitations; C++20 draft N4861 [vector.modifiers]; past-the-end wording

CPP-OBJ-05 Dangling std::string_view

SR-404 · Radio station display

× BUGGY
```
std::string_view station_name(int mhz) {
    std::string s = "KRAD " + std::to_string(mhz) + " FM STEREO";
    return s;              // view into a string destroyed on return
}
```

▸ ASan: heap-use-after-free

What goes wrong: This string_view refers to characters owned by the local string, whose lifetime ends at return. The view does not extend that lifetime. Invalidation can also happen before owner destruction, for example on storage reallocation; views into static backing storage have a different lifetime contract.

> FIX
```
std::string station_name(int mhz) {          // return the owner
    return "KRAD " + std::to_string(mhz) + " FM STEREO";
}
```

Avoid it: Author guidance: return an owning string here, or document and maintain the lifetime of borrowed storage. A view may be a valid return when its backing data outlives use. Compiler/sanitizer diagnostics vary by version, storage choice and path; the supplied host capture is not a promise that every dangling view is detected.

Sources: C++17 draft 24.4.2 [string.view.template]; 6.8 [basic.life]; 24.3.2 [basic.string]; Clang docs Introduction; Limitations

CPP-OBJ-06 Missing virtual destructor

SR-404 · ECU base class

✕ BUGGY

```
struct Ecu {
    virtual void tick() {}
};                                  // destructor is NOT
virtual
struct Radio : Ecu {
    std::string station = "KRAD 101.5 FM STEREO";
};
void scrap(Ecu* e) { delete e; }  // nonnull Radio*:
undefined in C++17
```

▸ `gcc -Wdelete-non-virtual-dtor: deleting object of polymorphic class …`

What goes wrong: In this C++17 example, deleting a nonnull Radio through Ecu* without a virtual base destructor is undefined. That rule does not specify which destructors physically run or require a particular sanitizer report. Deleting a null pointer is a separate permitted case.

> FIX

```
struct Ecu {
    virtual ~Ecu() = default;        // derived destructors
now run
    virtual void tick() {}
};
struct Radio : Ecu {
    std::string station = "KRAD 101.5 FM STEREO";
};
void scrap(Ecu* e) { delete e; }  // requires ownership and
a valid live object or nullptr
```

Avoid it: Author guidance: a base intended for polymorphic deletion can expose a public virtual destructor; protected nonvirtual destruction is another policy when base deletion must be forbidden. GCC14 documents selected deletion/class warnings. Owning smart pointers still need a valid deletion contract.

Sources: C++17 draft 8.3.5 [expr.delete]p3; 15.4 [class.dtor]; GCC14 docs -Wdelete-non-virtual-dtor; -Wnon-virtual-dtor; C++ Core Guidelines (guidance) C.35; GCC14 C++ options -Wdelete-non-virtual-dtor; -Wnon-virtual-dtor

CPP-OBJ-07 Throwing from a destructor

SR-404 · Trip log writer

✕ BUGGY

```
struct TripLog {
    bool flush() noexcept { return false; } // simulated
failure
    ~TripLog() {
        if (!flush())
            throw std::runtime_error("flush");  // →
std::terminate
    }
};
```

▸ `gcc -Wterminate: 'throw' will always call 'terminate'`

What goes wrong: This TripLog destructor has an implicit nonthrowing specification: its subobjects do not have potentially throwing destructors. Letting its exception escape calls terminate. Other destructors can have an implicit potentially throwing specification. An escaping destructor exception during exception unwinding also terminates.

> FIX

```
struct TripLog {
    bool flush() noexcept { return false; } // simulated
failure
    [[nodiscard]] bool close() noexcept {
        close_attempted_ = true; return flush();
    }
    ~TripLog() noexcept {
        if (!close_attempted_)
            (void)flush();  // simulated failure ignored;
no exception
    }
private:
    bool close_attempted_ = false;
};
```

Sources: C++17 draft 18.4 [except.spec]p8; 18.5.1 [except.terminate]; C++ Core Guidelines (guidance) E.16; GCC14 C++ options -Wterminate

Avoid it: Author guidance: report expected failure through an explicit operation and keep cleanup nonthrowing. Here flush is a simulated failure with no real I/O; close reports false, marks an attempt and prevents a destructor retry. It is not a durable file-commit API. GCC14 documents -Wterminate for selected escaping throws.

CPP-OBJ-08 Assuming moved-from contents are unchanged

SR-404 · RPM sample history

✕ BUGGY

```
std::vector<int> samples{3000, 3100, 3200};
store(std::move(samples));
if (samples.size() != 3) { // unjustified original-size
assumption
    std::cout << "original-size assumption failed\n";
    return 1;
}
```

▸ `prints: original-size assumption failed`

What goes wrong: std::move is a cast; the call's by-value construction performs the move. For standard-library types, absent a more specific contract, the source is valid with unspecified state. Calling vector::size after this move is valid. Assuming the old three elements remain is the error illustrated by the bad branch.

> FIX

```
std::vector<int> samples{3000, 3100, 3200};
std::cout << "before=" << samples.size() << '\n';
store(std::move(samples));
samples.clear(); // known empty state; queries were already
valid
samples.push_back(3300);
std::cout << "reused=" << samples.front() << '\n';
```

Avoid it: Author guidance: check each moved-from operation's preconditions. Queries such as size/empty are valid; front needs a nonempty vector. clear establishes an empty state before the shown reuse, but assignment/clear is not required before every operation. User-defined move behavior needs its own contract.

Sources: C++17 draft 23.2.5 [forward]; 20.5.5.15 [lib.types.movedfrom]; 20.3.25 [defns.valid]; 26.3.11 [vector]

CPP-OBJ-09 new[] paired with delete

SR-404 · Temperature table

✕ BUGGY

```
int* temps = new int[8]();
delete temps;             // must be delete[]: UB
```

▸ `gcc -Wmismatched-new-delete: 'void operator delete(void*, std::size_…`

What goes wrong: For this pointer from array new, a single-object delete violates C++17's deallocation contract. The language rule does not depend on an assumed allocation cookie or allocator layout. The other allocation/deallocation families also require their specified matching ownership and release operations.

> FIX

```
std::vector<int> temps(8);               // owns and sizes
itself
auto raw = std::make_unique<int[]>(8);  // if you need a
plain array
```

Avoid it: Author guidance: choose an array-owning vector, fixed-size array, or unique_ptr<T[]> where appropriate. The shown vector and make_unique array contain eight value-initialized ints. GCC14 documents -Wmismatched-new-delete in -Wall; sanitizer diagnostics remain tool/configuration-specific.

Sources: C++17 draft 8.3.5 [expr.delete]; 23.11.1.4 [unique.ptr.create]; 26.3.11.2 [vector.cons]; GCC14 docs -Wmismatched-new-delete; Clang docs Introduction; Usage; Limitations; GCC14 C++ options -Wmismatched-new-delete

CPP-OBJ-10 Reference to a temporary through a function

SR-404 · Cabin heater label

✕ BUGGY

```
const std::string& label(const std::string& s) { return s;
}
void show() {
    const std::string& t = label("CABIN HEATER MAX");  //
temp!
    std::cout << t << '\n';  // dead: it died at the ';'
above
}
```

▸ `gcc -Wdangling-reference: possibly dangling reference to a temporary`

What goes wrong: Here the converted string temporary bound to label's reference parameter lasts through the initializer's full-expression. Returning the reference does not extend it to t's lifetime, so the later stream access is invalid. Copying the referred string into an owning result before the full-expression ends is valid.

> FIX

```
const std::string& label(const std::string& s) { return s;
}
void show() {
    std::string t = label("CABIN HEATER MAX");  // owns a
copy
    std::cout << t << '\n';
}
```

Avoid it: Author guidance: use a value result when a borrowed reference may escape its backing lifetime. A function may safely return a reference to suitably long-lived input; min/max on temporaries are problematic when their returned reference is retained past the full-expression, not every immediate value use. Tool findings are bounded diagnostics.

Sources: C++17 draft 15.2 [class.temporary]p6; 28.7.8 [alg.min.max]; Clang docs Introduction; Limitations; GCC14 C++ options -Wdangling-reference

CPP-OBJ-11 shared_ptr reference cycle

SR-404 · Paired cabin ECUs

✕ BUGGY

```
struct Node {
    std::string name;
    std::shared_ptr<Node> peer;      // strong in both
directions
};
void pair_ecus() {
    auto a = std::make_shared<Node>(), b =
std::make_shared<Node>();
    a->peer = b;
    b->peer = a;  // strong cycle remains after these local
owners leave
}
```

▸ `LeakSanitizer: detected memory leaks`

What goes wrong: If pair_ecus completes normally with both strong peer links, releasing the local owners leaves each node owned by the other. This function does not break that cycle, so its nodes remain allocated. Reference cycles can be deliberately broken; forever and typical-frequency claims require separate context.

> FIX

```
struct Node {
    std::string name;
    std::weak_ptr<Node> peer;        // observes, doesn't
own
};
void pair_ecus() {
    auto a = std::make_shared<Node>(), b =
std::make_shared<Node>();
    a->peer = b;
    b->peer = a;                     // no strong peer
cycle
}
```

Avoid it: Author guidance: design explicit ownership and observation relationships. In this fixture weak peer links avoid a strong cycle, so both nodes are destroyed when the local owners leave scope. For later observation, lock a weak_ptr and check the returned shared_ptr before access; lock alone is not a liveness guarantee.

Sources: C++17 draft 23.11.2.2.2 [util.smartptr.shared.dest]; 23.11.2.3 [util.smartptr.weak]; 23.11.2.3.5 [util.smartptr.weak.obs]; Clang docs Memory leak detection; Limitations

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C++17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.

C++ Language Traps

Declaration rules, startup dependencies, proxies, deduction and concurrent access, with explicit version and evidence scope.

Debug build:
`-std=c++17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 11 entries

CPP-TRAP-01 Most vexing parse

SR-404 · Wiper timer

✕ BUGGY

```
Timer t();          // declares a FUNCTION returning Timer
t.ms = 500;         // error: 't' is not a class object
```

▸ `gcc -Wvexing-parse: empty parentheses were disambiguated as a functi…`

What goes wrong: In this declaration context Timer t() declares a function returning Timer, so t.ms is invalid. Engine e(Timer()) declares a function returning Engine with a parameter adjusted to a pointer to a function returning Timer. The language has specific ambiguity-resolution rules, not a universal rule for every token sequence.

✓ FIX

```
Timer t{};          // value-initialised object (or just:
Timer t;)
t.ms = 500;
```

Avoid it: Author guidance: use an unambiguous object initializer when an object is intended. Braces can choose initializer_list overloads or reject narrowing, so they are not interchangeable with every parenthesized initializer. GCC14 documents -Wvexing-parse; warning history and other compiler defaults need their own evidence.

Sources: C++17 draft 9.8 [stmt.ambig]; 11.2 [dcl.ambig.res]; 11.3.5 [dcl.fct]; GCC14 C++ options -Wvexing-parse

CPP-TRAP-02 Static initialisation order fiasco

SR-404 · Speedometer calibration

✕ BUGGY

```
// units.cpp
int mph_factor = read_calibration(); // may initialize
dynamically

// gauges.cpp
extern int mph_factor;
int g_scale = 2 * mph_factor;  // may observe only zero
initialization
```

▸ `ASan: initialization-order-fiasco`

What goes wrong: The shown ordinary globals are in different translation units and lack a single source-order guarantee across them. Static/zero initialization precedes dynamic initialization; ordering, permitted early initialization and deferral have separate N4659 rules. An early read can observe zero here; a physical crash for another class is not a required result.

✓ FIX

```
// units.h: keep runtime calibration, initialized at first
use
int read_calibration();
inline int mph_factor() {
    static const int f = read_calibration();
    return f;
}
// gauges.cpp: evaluate after runtime calibration is ready
int gauge_scale() { return 2 * mph_factor(); }
```

Avoid it: Author guidance: remove hidden startup dependencies. The local-static accessor initializes on first passage and supplies the fixed calibration16 before multiplication. Initialization exceptions are retried on later passage; recursive reentry during initialization is invalid. ASan's captured initialization-order report is an instrumented host observation, not a universal C++ outcome.

Sources: C++17 draft 6.6.2 [basic.start.static]; 6.6.3 [basic.start.dynamic]; 9.7 [stmt.dcl]; Clang docs Initialization order checking; Limitations

CPP-TRAP-03 ODR violation: same inline name, different body

SR-404 · Speedometer vs. tachometer

✕ BUGGY

```
// speedo.cpp
inline int scale() { return 1; }     // km/h per tick

// tacho.cpp
inline int scale() { return 100; }   // rpm: same name,
other body
```

▸ `compiled/linked rc=0; not executed: ODR mismatch: no diagnostic requ…`

What goes wrong: These two external inline definitions name the same entity but use different tokens, violating N4659's multiple-definition conditions. That cross-unit case requires no diagnostic. Token identity alone is not sufficient: name lookup and other ODR conditions also matter. The invalid program has no promised linker choice or output.

✓ FIX

```
// speedo.cpp
namespace { int scale() { return 1; } }     // internal
linkage

// tacho.cpp
namespace { int scale() { return 100; } }   // a different
function
```

Avoid it: Author guidance: use internal linkage for genuinely independent file-local helpers, and maintain consistent tokens and required lookup bindings for shared definitions. The bad example is compiled/linked only, with its raw result retained; it is not executed to invent a portable outcome. The fixed independent helpers are actually run.

Sources: C++17 draft 6.2 [basic.def.odr]p6; 10.3.1 [namespace.def]

CPP-TRAP-04 Implicit conversion via a non-explicit constructor

SR-404 · Rev limiter

✕ BUGGY

```
struct Rpm {
    Rpm(int v) : value(v) {}      // converting constructor
    int value;
};
void set_redline(Rpm r) { std::printf("redline %d\n",
r.value); }
// set_redline(true);  compiles: bool → int → Rpm{1}
```

▸ `prints: redline 1`

What goes wrong: The non-explicit int constructor permits a standard bool-to-int conversion followed by construction of Rpm; the supplied true input produces value1. This demonstrates an admitted conversion, not validated rev-limit units or sensor range. Constructor/conversion behavior depends on the actual overloads and initialization context.

✓ FIX

```
struct Rpm {
    explicit Rpm(int v) : value(v) {}
    int value;
};
void set_redline(Rpm r) { std::printf("redline %d\n",
r.value); }
// set_redline(true);       error: no implicit conversion
// set_redline(Rpm{7000});  intent is visible
```

Avoid it: Author guidance: choose explicit constructors for APIs where implicit conversion is unintended. Explicit still permits direct initialization and does not validate integer range; Rpm{true} remains possible here. Conversion to bool has contextual exceptions for explicit conversion functions. Conditional explicit is a C++20 facility, not this C++17 fix.

Sources: C++17 draft 15.3.1 [class.conv.ctor]; 7.14 [conv.bool]; 16.3.3.1 [over.best.ics]; 7 [conv]p4; 15.3.2 [class.conv.fct]; C++20 draft N4861 [dcl.fct.spec]: conditional explicit

CPP-TRAP-05 std::vector<bool> is not a container of bool

SR-404 · Dash warning lamps

✕ BUGGY

```
std::vector<bool> lamps(4);
bool& first = lamps[0];  // cannot bind this proxy to
nonconst bool&
```

▸ `gcc error: cannot bind non-const lvalue reference of type 'bool&' to…`

What goes wrong: N4659 vector<bool>::reference is a proxy, so the shown nonconst bool& binding is invalid. auto x=v[0] keeps that proxy; bool b=v[0] copies a value. Packed bits are recommended, not a mandated allocator layout. The standard excludes vector<bool> from the different-element concurrent-modification guarantee; not every pair of writes must race.

✓ FIX

```
std::vector<char> lamps(4);   // real, addressable elements
char& first = lamps[0];       // an addressable char
element
first = 1;
```

Avoid it: Author guidance: use an addressable element type such as char when real element references are needed. No data() member is specified for this N4659 specialization. A proxy must still obey backing-storage validity; use an explicit bool value when a snapshot is wanted. bitset has its own proxy behavior and is not a universal bool-reference replacement.

Sources: C++17 draft 26.3.12 [vector.bool]; 26.2.2 [container.requirements.dataraces]; 23.9.2 [template.bitset]

CPP-TRAP-06 auto deduction surprises

SR-404 · Trip computer tables

✕ BUGGY
```cpp
const std::vector<int>& readings();  // this fixture has three elements
void scan() {
    auto r = readings();   // std::vector<int>: a full COPY
    auto x = {5};          // std::initializer_list<int>, not int
    std::cout << r.size() + x.size() << '\n';
}
```

▸ prints: 4

What goes wrong: For this ordinary by-value auto declaration, deduction removes the initializer's reference/top-level const and copies the lvalue vector. Other initializers can move or elide. Under N4659 auto x={5} deduces initializer_list<int>, while auto y{5} deduces int. N3922 is defect wording, not proof that every earlier language mode must differ.

> FIX
```cpp
const std::vector<int>& readings();  // this fixture has three elements
void scan() {
    const auto& r = readings();  // binds to the table: no copy
    auto x = 5;  // int (auto x{5} is also int in C++17)
    std::cout << r.size() + x << '\n';
}
```

Avoid it: Author guidance: spell the intended reference/value policy and check deduced types. The fixed const auto& borrows the live static table; its lifetime is part of the contract. Modern compilers may apply defect changes in earlier modes. Avoiding every brace is a style choice, not a language requirement; the actual hidden type checks are called in this candidate.

Sources: C++17 draft 10.1.7.4 [dcl.spec.auto]; 10.1.7.4.1 [dcl.type.auto.deduct]; WG21 N3922 (defect wording) Background; Summary of Proposed Changes

CPP-TRAP-07 Lambda captures a local by reference, then outlives it

SR-404 · Tachometer callback

✕ BUGGY
```cpp
std::function<int()> make_reader() {
    int rpm = 3000;
    return [&] { return rpm; };   // rpm dies on return
}
```

▸ ASan: stack-use-after-return

What goes wrong: The returned lambda refers to the automatic int rpm after its lifetime ends. Invoking that access is invalid; the language does not mandate a physical dead stack slot or particular printed value. A lambda can outlive a capture's declaring scope safely when the referenced backing object remains live.

> FIX
```cpp
std::function<int()> make_reader() {
    int rpm = 3000;
    return [rpm] { return rpm; };  // copy into the closure
}
```

Avoid it: Author guidance: choose capture ownership deliberately. Copying a pointer/view does not extend its backing lifetime. A move-only init-capture can make the closure move-only; N4659 std::function requires a copy-constructible target. The shown captured int is copyable. ASan's return-lifetime capture has instrumentation/platform scope.

Sources: C++17 draft 8.1.5.2 [expr.prim.lambda.capture]; 6.8 [basic.life]; 23.14.13.2.1 [func.wrap.func.con]; C++ Core Guidelines (guidance) F.53 (design guidance); Clang docs Stack use after return; Limitations

CPP-TRAP-08 Data race on a shared counter

SR-404 · Wheel-tick counter

✕ BUGGY
```cpp
unsigned long ticks = 0;
std::thread t([&] { for (unsigned long i = 0; i < 100000UL; ++i) ++ticks; });
for (unsigned long i = 0; i < 100000UL; ++i) ++ticks;  // unsynchronised: UB
t.join();
std::cout << ticks << '\n';  // TSan capture; no required ordinary output
```

▸ TSan: data race

What goes wrong: The two unsynchronized increments conflict on the same non-atomic memory location without a happens-before relation: a C++17 data race. Undefined behavior does not guarantee lost updates, torn values or a particular count. The fixed unsigned-long atomic count uses defined RMW operations and representable loop/count values.

> FIX
```cpp
std::atomic<unsigned long> ticks{0};
std::thread t([&] { for (unsigned long i = 0; i < 100000UL; ++i) ++ticks; });
for (unsigned long i = 0; i < 100000UL; ++i) ++ticks;  // atomic RMW
t.join();
std::cout << ticks << '\n';  // 200000 after both loops complete and join returns
```

Avoid it: Author guidance: use a suitable atomic or a shared locking protocol for these accesses; atomicity of a counter is not a complete publication/lifetime protocol for other data. After both100000-increment loops complete and join returns, the fixed count is200000. Lock freedom is not promised. TSan checks executed instrumented paths and has tool/platform limits.

Sources: C++17 draft 4.7 [intro.races]; 6.9.1 [basic.fundamental]; 32.6 [atomics.types.generic]; 33.3.2.5 [thread.thread.member]; Clang ThreadSanitizer docs Usage; Limitations

CPP-TRAP-09 Virtual call from a constructor

SR-404 · Brakes ECU init

✕ BUGGY
```cpp
struct Ecu {
    Ecu() { init(); }  // runs Ecu::init, not the override
    virtual ~Ecu() = default;
    virtual void init() { std::puts("generic init"); }
};
struct Brakes : Ecu {
    void init() override { std::puts("ABS init"); }
};
```

▸ prints: generic init

What goes wrong: For the call on the object currently in Ecu's constructor, virtual dispatch selects the final overrider in that constructor's class, giving generic init. Construction/destruction has specific dispatch rules; calls on other complete objects differ. A virtual call to a pure virtual for that construction/destruction case is undefined, not a required abort.

> FIX
```cpp
struct Ecu {
    virtual ~Ecu() = default;
    virtual void init() { std::puts("generic init"); }
};
struct Brakes : Ecu {
    void init() override { std::puts("ABS init"); }
};
template <class T> std::unique_ptr<T> make_ecu() {
    auto e = std::make_unique<T>();
    e->init();  // fully constructed: dispatches
    return e;
}
```

Sources: C++17 draft 15.7 [class.cdtor]p4; 13.4 [class.abstract]p6; 23.11.1.2.2 [unique.ptr.single.dtor]

Avoid it: Author guidance: choose an initialization API that avoids depending on a not-yet-constructed derived part. This factory calls init only after T is constructed and retains ownership if init throws, but does not roll back arbitrary external side effects. Virtual calls in constructors are not all forbidden by the language.

CPP-TRAP-10 map::operator[] inserts on lookup

SR-404 · Speed-limit table

✕ BUGGY
```cpp
std::map<std::string, int> limits{{"rpm", 6500}};
if (limits["speed"] > 80)           // inserts {"speed", 0}!
    std::puts("too fast");
std::cout << limits.size() << '\n'; // 2
```

▸ prints: 2

What goes wrong: For the absent speed key in this N4659 map, operator[] inserts an entry whose int mapped value is initialized to zero. An existing key does not cause a second insertion. These associative containers have no const operator[] overload in this version; the supplied GCC negative case checks that expression.

> FIX
```cpp
std::map<std::string, int> limits{{"rpm", 6500}};
auto it = limits.find("speed");
if (it != limits.end() && it->second > 80)
    std::puts("too fast");
std::cout << limits.size() << '\n'; // 1
```

Avoid it: Author guidance: choose find for an explicit missing-key branch or at for checked access that throws out_of_range on absence. C++20 contains is a separate facility. Protect shared mutations and follow container data-race contracts; insertion-capable lookup is not a general unsynchronized-read API. The fixed find path leaves this map unchanged.

Sources: C++17 draft 26.4.4.3 [map.access]; 26.2.2 [container.requirements.dataraces]; 26.2.6 [associative.reqmts]; C++20 draft N4861 [associative.reqmts]; contains

CPP-TRAP-11 std::move on a const object copies

SR-404 · Trip computer name

✕ BUGGY

```
const std::string name = "SR-404 TRIP COMPUTER";
std::string dst = std::move(name);   // const&& → copy constructor
std::cout << name.size() << '\n';  // copy selected by these string overloads
```

▸ prints: 20

What goes wrong: Here moving the const string yields const string&&, which cannot bind the ordinary string&& move constructor, so the string copy overload is selected. Other user types can provide different overloads. Equal printed lengths do not by themselves prove copying or buffer transfer. An eligible named local returned as std::move(local) loses NRVO eligibility.

✓ FIX

```
std::string name = "SR-404 TRIP COMPUTER";  // non-const: movable
std::string dst = std::move(name);  // move constructor selected
std::cout << dst.size() << '\n';
```

Avoid it: Author guidance: use const/value/move according to the intended API rather than applying std::move mechanically. Return an otherwise eligible local by name when NRVO/implicit move is intended. GCC14 documents selected pessimizing/redundant-move warnings. The nonconst string fix selects its move constructor, without promising a particular storage transfer or moved-from value.

Sources: C++17 draft 23.2.5 [forward]; 24.3.2.2 [string.cons]; 15.8.3 [class.copy.elision]; GCC14 C++ options -Wpessimizing-move; -Wredundant-move

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C++17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.

SHEET 07

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

Concurrency & Embedded

Fictional ECU-themed examples of C concurrency, POSIX signals and bounded numerical/input policies. Language rules, platform contracts and recorded tool observations have separate scopes.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 10 entries

CON-01 Race condition: check-then-act

SR-404 · Fuel pump (two injector tasks)

✕ BUGGY

```c
static int fuel_ml = 100;
void *pump(void *arg) {
    if (fuel_ml >= 60)          // check ...
        fuel_ml -= 60;  // conflicting access: data race
    return arg;
}
```

▶ TSan: data race

What goes wrong: The two threads access the same non-atomic fuel_ml with conflicting operations and no synchronization: a C11 data race and undefined behavior. Neither a final value of -20 nor lost updates is guaranteed. Replacing separate accesses with atomics can remove that data race while still allowing both tasks to pass the check.

> FIX

```c
static int fuel_ml = 100;
static pthread_mutex_t fuel_lock =
PTHREAD_MUTEX_INITIALIZER;
void *pump(void *arg) {
    check_pthread(pthread_mutex_lock(&fuel_lock));
    if (fuel_ml >= 60)          // check and act under ONE
lock
        fuel_ml -= 60;
    check_pthread(pthread_mutex_unlock(&fuel_lock));
    return arg;
}
```

Avoid it: Author guidance: protect the complete invariant with one mutex, or design a suitable compare-exchange operation with failure handling. The linked driver checks pthread return codes. TSan can diagnose an exercised data race; a clean run is not proof that every interleaving or logic error is safe.

Sources: C11 draft §5.1.2.4 ¶25; §7.17.3; §7.17.7.4; POSIX pthread mutex DESCRIPTION and return values; POSIX2024 RETURN VALUE and error handling; POSIX2024 DESCRIPTION and return values; Clang ThreadSanitizer docs Introduction; usage and detection limits

CON-02 Deadlock from inconsistent lock order

SR-404 · Bus + dashboard locks

✕ BUGGY

```c
void *gauge_task(void *p);      // locks bus, then dash
void *alarm_task(void *p) {
    check_pthread(pthread_mutex_lock(&dash));  // dash,
then bus: opposite order
    check_pthread(pthread_mutex_lock(&bus));
    check_pthread(pthread_mutex_unlock(&bus));
    check_pthread(pthread_mutex_unlock(&dash));
    return p;
}
```

▶ TSan: lock-order-inversion (potential deadlock)

What goes wrong: Taking bus then dash in one task and dash then bus in another permits a circular wait if both run concurrently and each holds its first mutex. This supplied driver actually joins the first task before starting the second: it demonstrates a lock-order detector report, not an observed deadlock or an elapsed-time prediction.

> FIX

```c
void *gauge_task(void *p);      // locks bus, then dash
void *alarm_task(void *p) {     // documented order: bus,
then dash
    check_pthread(pthread_mutex_lock(&bus));
    check_pthread(pthread_mutex_lock(&dash));
    check_pthread(pthread_mutex_unlock(&dash));
    check_pthread(pthread_mutex_unlock(&bus));
    return p;
}
```

Avoid it: Author guidance: define and follow an acyclic lock order, limit critical sections according to requirements, and audit calls that may acquire locks. C++17 scoped_lock uses the standard multiple-lock algorithm, but cannot rule out every unrelated lock cycle in a program. The observed TSan lock-order report is tool-specific and can occur without an actual deadlock.

Sources: POSIX pthread mutex blocking and mutex types; return values; POSIX2024 RETURN VALUE; POSIX2024 DESCRIPTION: wait for termination; C++17 draft [thread.lock.algorithm]; [thread.lock.scoped]; Sanitizer runtime docs detect_deadlocks; observed report is separately bound to the supplied run

CON-03 volatile used as if it were atomic

SR-404 · Wheel-tick counter

✕ BUGGY

```c
static volatile unsigned long ticks;  // volatile is NOT
thread-safe
void *count_ticks(void *p) {
    for (unsigned long i = 0; i < 100000UL; i++)
        ticks++;  // concurrent non-atomic increment: data
race
    return p;
}
```

▶ TSan: data race

What goes wrong: volatile access semantics do not provide the atomicity or inter-thread synchronization needed by ticks++. These conflicting non-atomic accesses form a C11 data race, with undefined behavior rather than a guaranteed lost-count result. What constitutes a volatile access is implementation-defined.

> FIX

```c
static atomic_ulong ticks;        // <stdatomic.h>, C11
void *count_ticks(void *p) {
    for (unsigned long i = 0; i < 100000UL; i++)
        atomic_fetch_add_explicit(&ticks, 1,
memory_order_relaxed);
    return p;
}
```

Avoid it: Author guidance: use atomic operations for the shared counter and an appropriate synchronization design for compound state. Relaxed fetch-add supplies an indivisible counter update here, but does not publish other data or guarantee lock-free implementation. Volatile signal flags have narrow signal rules; hardware-register access needs target-specific contracts, not an "only these uses" language rule.

Sources: C11 draft §5.1.2.4 ¶25; §6.7.3 ¶7; §7.17.3; §7.17.7.5; §7.14.1.1 ¶5; §5.2.4.2.1 minimum unsigned-long range; §7.21.6.1 %lu; GCC volatile documentation implementation-defined accesses; volatile is not a non-volatile-memory barrier; Clang ThreadSanitizer docs data-race detection, not proof of race freedom

CON-04 Unsafe calls inside a POSIX signal handler

SR-404 · Timer-signal exercise, not a hardware ISR

✕ BUGGY

```c
static void on_tick(int sig) {
    printf("tick %d\n", sig);   // not POSIX async-signal-
safe
}
void start_ticks(void) {
    if (signal(SIGALRM, on_tick) == SIG_ERR) {
        perror("signal setup"); exit(EXIT_FAILURE);
    }
    alarm(1);
}
```

▶ gcc -Wanalyzer-unsafe-call-within-signal-handler: call to 'printf' f…

What goes wrong: This example uses a POSIX SIGALRM handler, not a hardware interrupt implementation. printf is not async-signal-safe; calling it from an asynchronously delivered handler can violate the signal-safety contract. Deadlock or corruption is not a guaranteed result, and hardware interrupt latency depends on a separate target contract.

> FIX

```c
static volatile sig_atomic_t tick_pending;
static void on_tick(int sig) {
    (void)sig;
    tick_pending = 1;  // coalesces events; not an event
count
}
/* Install with checked sigaction in the linked fixed
setup.
   Main loop handles the pending flag outside the handler.
*/
```

Avoid it: Author guidance: defer ordinary I/O to normal execution and follow the applicable signal-safe operation rules. POSIX lists write as async-signal-safe, but its descriptor, return value and errno still require care. The volatile sig_atomic_t assignment is a pending-event flag, not a reliable event counter or an arbitrary-thread synchronization mechanism. GCC analyzer diagnosis is a bounded tool observation.

Sources: C11 draft §7.14.1.1 ¶5: asynchronous handler restrictions; POSIX signal rules §2.4.3 Signal Actions: async-signal-safe operation list; POSIX signal signal return value SIG_ERR and POSIX interface scope; POSIX sigaction DESCRIPTION and RETURN VALUE; POSIX sigemptyset DESCRIPTION and RETURN VALUE; POSIX alarm scheduled signal and remaining-time return value; POSIX2024 RETURN VALUE and error conditions; GCC14 docs -Wanalyzer-unsafe-call-within-signal-handler; compiler diagnosis scope

CON-05 Publishing data with relaxed atomics

SR-404 · Sensor frame hand-off

✕ BUGGY

```
void *producer(void *p) {
    frame[0] = 42;
    atomic_store_explicit(&ready, true,
                          memory_order_relaxed);   // no
ordering!
    return p;
}
void *consumer(void *p) {
    while (!atomic_load_explicit(&ready,
memory_order_relaxed)) { }
    printf("frame %d\n", frame[0]);   // stale? data race!
    return p;
}
```

▸ TSan: data race

What goes wrong: The relaxed flag operations are atomic but do not synchronize publication of the ordinary frame element. The producer write and consumer read are not ordered by happens-before, so this program has a C11 data race and undefined behavior. This diagnosis does not depend on observing a stale value on ARM or any other CPU.

> FIX

```
void *producer(void *p) {
    frame[0] = 42;
    atomic_store_explicit(&ready, true,
                          memory_order_release);   //
publish
    return p;
}
void *consumer(void *p) {
    while (!atomic_load_explicit(&ready,
memory_order_acquire)) { }
    printf("frame %d\n", frame[0]);  // after observing
this release: value is 42
    return p;
}
```

Avoid it: Author guidance: for this one-shot example, a release store and an acquire load that reads from it establish the synchronization needed for frame[0]. Reusing a buffer needs an additional ownership/protocol design. Neither this spin loop nor the language promises a response deadline; TSan observations do not prove all schedules.

Sources: C11 draft §5.1.2.4 ¶11, ¶25; §7.17.3: relaxed vs release/acquire; observation of release; POSIX2024 RETURN VALUE; POSIX2024 join contract; Clang ThreadSanitizer docs instrumented race detection and limits

CON-06 Unbounded wait: the watchdog bites

SR-404 · Wheel-speed sensor + ECU watchdog

✕ BUGGY

```
bool sensor_ready(void);  // this demonstration supplies an
always-false stub
int read_wheel_speed(void) {
    while (!sensor_ready()) { }   // no retry bound in this
example
    return sensor_value();
}
```

▸ hangs: killed by the test watchdog

What goes wrong: The supplied sensor stub always returns false, so the bad function does not return in the host demonstration and is killed by the test runner's timeout. Separately, the game ECU watch() faults a busy running worker after three consecutive checks with elapsed time greater than 500 ms; boot uses 5000 ms. Those are game rules, not a precise 500-ms hardware deadline.

> FIX

```
bool sensor_ready(void);
int read_wheel_speed(int *out) {
    if (out == NULL) return -1;
    for (int tries = 0; tries < 1000; tries++) {
        if (sensor_ready()) { *out = sensor_value(); return
0; }
    }
    return -1;  // caller decides how to handle the fault
}
```

Avoid it: Author guidance: define error handling and suitable time bounds for the actual system, including each called operation. This retry loop bounds calls to sensor_ready, not wall-clock duration if that function can block. Returning -1 reports a fault; the caller must implement any fallback. Hardware watchdog service policy needs its actual platform and supervision requirements; use fault injection to test them.

Sources: C11 draft §6.8.5; §6.5.2.2; §6.8.6.4: control flow is not a wall-clock bound; Game ECU runtime WATCHDOG_MS/BOOT_MS; watch >limit and strikes>=3; start timer; _haywire/_kill

CON-07 No input validation or output clamping

SR-404 · Steering actuator

✕ BUGGY

```
float rate_cmd(float wheel, float dt) {
    return wheel / dt * 0.05f;  // shown Annex F samples:
+inf and NaN
}
```

▸ prints: inf nan

What goes wrong: For the stated Annex F build with default non-trapping exceptions, the supplied positive numerator divided by +0 produces infinity and the NaN input produces a NaN result. These are not universal C11-core arithmetic outcomes. The game actuator separately replaces every non-finite scalar output with zero and clamps to the system range; source code alone does not establish real vehicle effects.

> FIX

```
float rate_cmd(float wheel, float dt) {
    if (!isfinite(wheel) || !isfinite(dt) || !(dt > 0.0f))
        return 0.0f;  // exercise policy, not a safety
proof
    float scaled = wheel * 0.05f;
    if (scaled >= dt) return 1.0f;
    if (scaled <= -dt) return -1.0f;
    return scaled / dt;  // magnitude below 1 before
division
}
```

Avoid it: Author guidance: define acceptable inputs and an explicit application output policy. This Annex F example rejects non-finite inputs and non-positive dt, then scales before division and saturates before a potentially overflowing quotient. Tiny inputs can still round or underflow. Build assumptions exclude fast-math and changed floating-point environment modes. A numerical bound or zero default is not evidence of physical safety.

Sources: C11 draft §6.5 ¶5 and §6.5.5 vs Annex F.1/F.3; F.8.3 default exception environment; §7.12.3.2 isfinite; GCC14 docs -ffast-math; -ffinite-math-only; incompatible assumptions excluded here; Game ECU runtime RANGE and _actuator: non-finite scalar outputs become zero and other outputs are bounded

CON-08 Trusting a length field (fuzz your parsers)

SR-404 · CAN / diagnostics-port frames

✕ BUGGY

```
int parse_frame(const uint8_t *buf, size_t n, uint8_t
out[8]) {
    uint8_t len = buf[0];          // comes off the wire
    memcpy(out, buf + 1, len);     // len > 8 or > n - 1:
overflow
    return len;
}
```

▸ ASan: heap-buffer-overflow

libFuzzer harness · instrumented exploration; no discovery-time guarantee

FUZZ

```
int LLVMFuzzerTestOneInput(const uint8_t *d, size_t n) {
    uint8_t out[8];
    if (parse_frame(d, n, out) > 0) {
        volatile uint8_t sink = out[0];  // keep the copy
observable
        (void)sink;
    }
    return 0;
}
```

What goes wrong: Reading buf[0] requires an available input byte. A length exceeding the readable payload or writable output capacity makes the copy invalid; the supplied 200-byte request exceeds both example objects, but no particular failure or diagnosis is guaranteed by C. The array parameter syntax out[8] adjusts to a pointer and does not enforce capacity.

> FIX

```
int parse_frame(const uint8_t *buf, size_t n, uint8_t
out[8]) {
    if (n < 1 || buf == NULL || out == NULL) return -1;
    uint8_t len = buf[0];
    if (len > 8 || len > n - 1) return -1;   // validate
first
    memcpy(out, buf + 1, len);
    return len;
}
```

Avoid it: Author guidance: check lengths before access and document pointer, capacity and non-overlap contracts. The fixed API requires out to provide eight writable bytes and buf to provide the claimed readable bytes, with no overlap for memcpy; integer n alone cannot prove those properties. libFuzzer explores supplied instrumented builds and corpora, with no guaranteed discovery time or exhaustive correctness proof.

Sources: C11 draft §6.7.6.3 ¶7 array parameter adjustment; §6.5.2.1; §7.24.2.1 memcpy contracts; Clang docs instrumented out-of-bounds detection and limitations; Clang libFuzzer docs in-process input exploration and corpus; no time/completeness guarantee

CON-09 Condition variable waited with if, not while

SR-404 · Frame-ready hand-off

✕ BUGGY

```c
void *ecu_consumer(void *p) {
    check_pthread(pthread_mutex_lock(&m));
    if (!frame_ready)
        wait_for_frame();  // checked cond_wait; witness in scaffold
    printf("frame=%d\n", frame);  // predicate may still be false
    check_pthread(pthread_mutex_unlock(&m));
    return p;
}
```

▸ prints: frame=0

What goes wrong: POSIX condition waits may wake spuriously, and a signaled wakeup does not itself establish the application predicate. The supplied driver deliberately signals while frame_ready is false; that is an explicit signal, not an observed spontaneous spurious wakeup. Separate witness coordination replaces timing sleeps.

> FIX

```c
void *ecu_consumer(void *p) {
    check_pthread(pthread_mutex_lock(&m));
    while (!frame_ready)
        wait_for_frame();  // checked cond_wait; witness in scaffold
    printf("frame=%d\n", frame);  // predicate holds with the mutex locked
    check_pthread(pthread_mutex_unlock(&m));
    return p;
}
```

Avoid it: Author guidance: test the predicate under the associated mutex after every successful wakeup and handle errors; C++ predicate wait overloads perform their own loop. This linked driver uses a second condition variable only to coordinate the demonstration. Correct predicate use does not by itself guarantee scheduler progress, producer completion or bounded waiting.

Sources: POSIX condition waits DESCRIPTION: predicate, spurious wakeups, mutex association; RETURN VALUE; POSIX pthread mutex lock/unlock contracts; POSIX2024 return value handling; POSIX2024 join contract; C++17 draft [thread.condition.condvar]: predicate wait overload

CON-10 Tick-counter wraparound in timeouts

SR-404 · ECU watchdog timer

✕ BUGGY

```c
bool timed_out(uint32_t now_ms, uint32_t start_ms) {
    uint32_t deadline = start_ms + 500;  // deadline addition can wrap near UINT32_MAX
    return now_ms >= deadline;  // wrong even before now itself wraps
}
```

▸ prints: timed_out=1 after 50 ms

What goes wrong: An exact 32-bit millisecond counter has period 2^32 ms, about 49.7103 days. The shown start is 100 ticks below UINT32_MAX: adding 500 wraps the deadline, so the bad comparison already reports timeout after only 50 ms, before the counter itself wraps. Unsigned elapsed subtraction represents elapsed time only within a known single-period horizon.

> FIX

```c
bool timed_out(uint32_t now_ms, uint32_t start_ms) {
    return (uint32_t)(now_ms - start_ms) >= 500u;  // one-wrap horizon
}
```

Avoid it: Author guidance: validate that the true elapsed duration is less than 2^32 ticks and poll in time to observe the required timeout. Multiple elapsed periods are indistinguishable from these two timestamps. A wider counter or a supported monotonic clock API may suit longer spans, but clock_gettime does not promise a 64-bit application counter; check return values, clock support, representation and units.

Sources: C11 draft §6.2.5 ¶9; §6.3.1.3; §7.20.1.1: exact-width unsigned type, modulo conversion; 2^32/86400000≈49.7103 is derived; POSIX clock_gettime CLOCK_MONOTONIC clock semantics, timespec, RETURN VALUE and errors

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.